



3DEXPERIENCE®

Dymola培训

达索系统 CATIA

培训内容(1/3)

❖ 系统建模与仿真

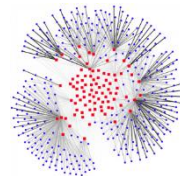
- ❖ 什么是系统，为什么要仿真
- ❖ 仿真模型形式
- ❖ 仿真模型范例

❖ Modelica和Dymola概述

- ❖ 什么是Modelica语言
- ❖ 什么是Dymola软件
- ❖ 两者如何搭配工作
- ❖ 练习一：工业机器人

❖ Modelica语言初探

- ❖ Classes 类
- ❖ Types and Declaration 类型和声明
- ❖ Units 单位
- ❖ Inheritance 继承
- ❖ 练习二：单摆
- ❖ 练习三：月球登陆器
- ❖ Connectors 接口



Language Specification

Version 3.3

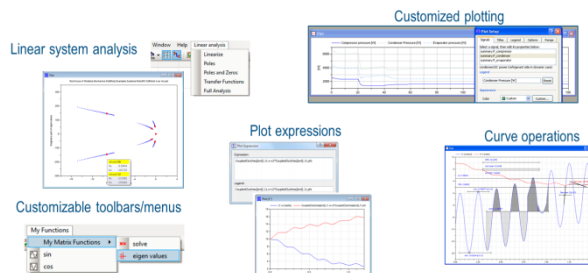
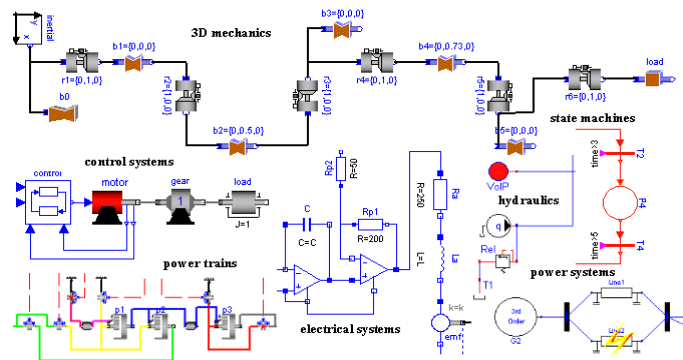
培训内容(2/3)

❖ Modelica语言初探 (续)

- ❖ 练习四：雨刮器
- ❖ Acausality 非因果性
- ❖ 练习五：直流电机
- ❖ Discrete Events 离散事件
- ❖ Hybrid Modeling 混合建模
- ❖ 练习六：受控直流电机
- ❖ 练习七：温度控制
- ❖ Arrays 数组
- ❖ 练习八：振动梁和热传导

❖ Dymola仿真环境

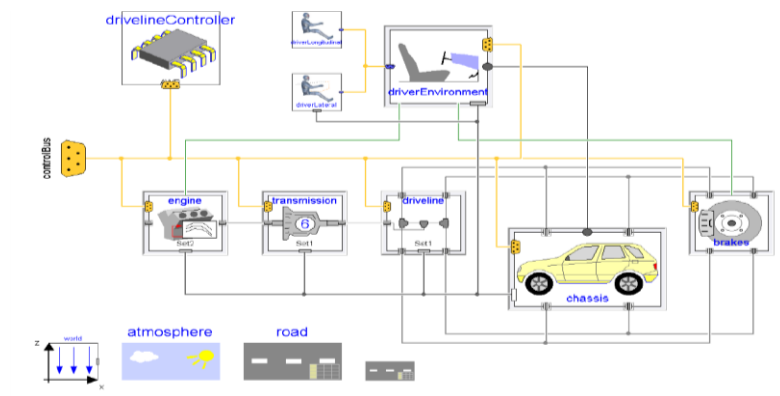
- ❖ 建模窗口
- ❖ 文本编辑窗口
- ❖ 浏览及生成文档报告
- ❖ 仿真窗口
- ❖ 基础仿真脚本
- ❖ 练习九：RV-2AJ 机器人



培训内容(3/3)

❖ Modelica/Dymola进阶技巧

- ❖ 用Dymola进行设计优化/DOE/实验分析
- ❖ 练习十：优化控制器
- ❖ Records 数据表
- ❖ 练习十一：Records 数据表
- ❖ Redclaration 重声明
- ❖ 练习十二：生成模版
- ❖ Inner和Outer前缀
- ❖ 动态显示图标
- ❖ 调用外部函数
- ❖ Expandable Connectors 可扩展接口
- ❖ 练习十三：可扩展接口



培训内容

❖ 系统建模与仿真

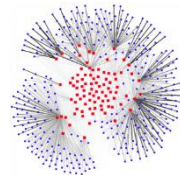
- ❖ 什么是系统，为什么要仿真
- ❖ 仿真模型形式
- ❖ 仿真模型范例

❖ Modelica和Dymola概述

- ❖ 什么是Modelica语言
- ❖ 什么是Dymola软件
- ❖ 两者如何搭配工作
- ❖ 练习一：工业机器人

❖ Modelica语言初探

- ❖ Classes 类
- ❖ Types and Declaration 类型和声明
- ❖ Units 单位
- ❖ Inheritance 继承
- ❖ 练习二：单摆
- ❖ 练习三：月球登陆器
- ❖ Connectors 接口



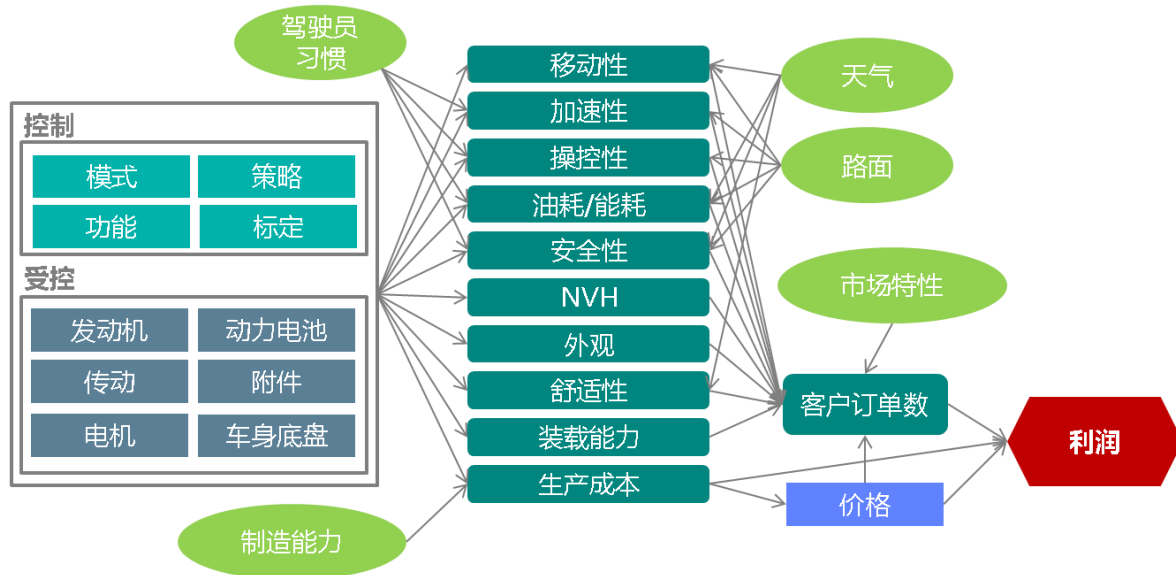
什么是系统，为什么要仿真(1/2)

ISO/IEC15288:2008: "A combination of interacting elements organized to achieve one or more stated purposes."

INCOSE: A construct or collection of different elements that together produce results not obtainable by the elements alone.

研究系统的目的

- ❖ 确认需求
- ❖ 理解互动
- ❖ 指导设计
- ❖ 提高质量



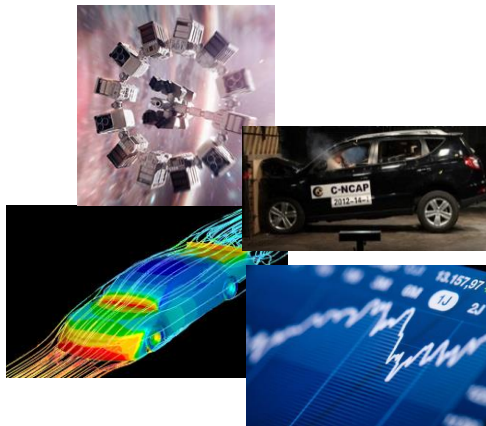
什么是系统，为什么要仿真(2/2)

如何研究系统

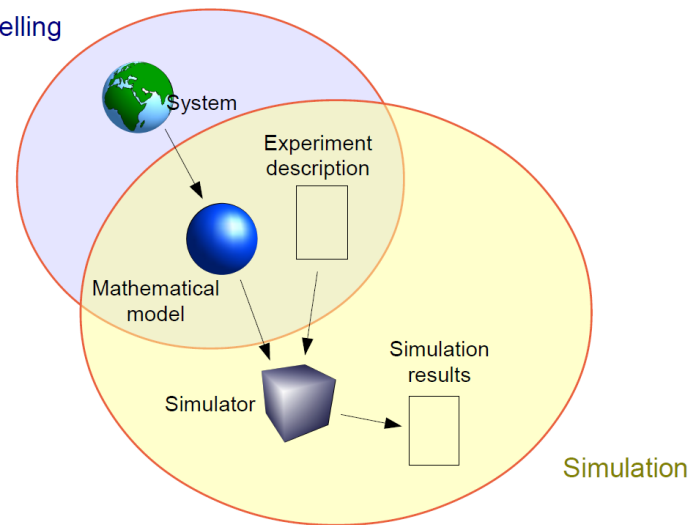
- ❖ 在真实系统上直接进行实验
- ❖ 将真实系统抽象成模型然后仿真

为什么要仿真

- ❖ 无法进行真实实验
 - ❖ 真实系统仍未存在
 - ❖ 实验风险太大
 - ❖ 实验成本太高
- ❖ 无法进行分析求解
 - ❖ 不确定性
 - ❖ 动态性太强
 - ❖ 过于复杂



Modelling



仿真模型形式(1/10)

Context 上下文：系统所在的，对系统有影响的外部环境，例如温度、压力、空间限制...等等



仿真模型形式(2/10)

Analog(木条+绳索)



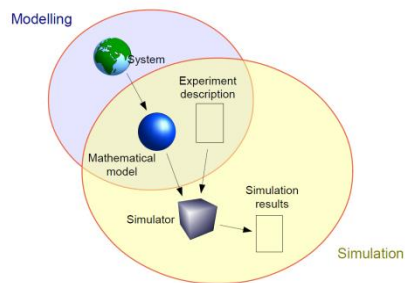
Antoinette Trainer ~1910

Digital(Dymola + Mechatronics)



DLR 德宇航 2013

仿真模型形式(3/10)

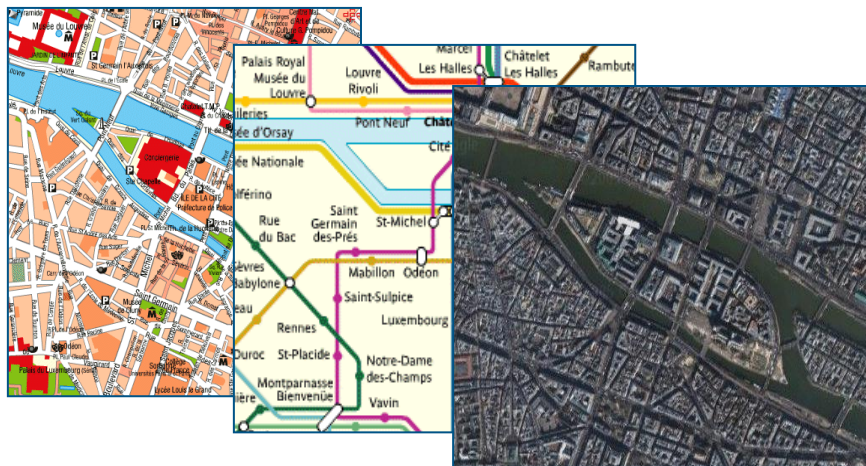


Stakeholder

Point of view

View and concern

Real world



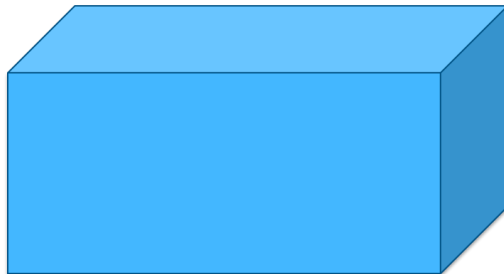
仿真模型形式(4/10)

“工程的”，“统计的”，“经验的”



Mouse

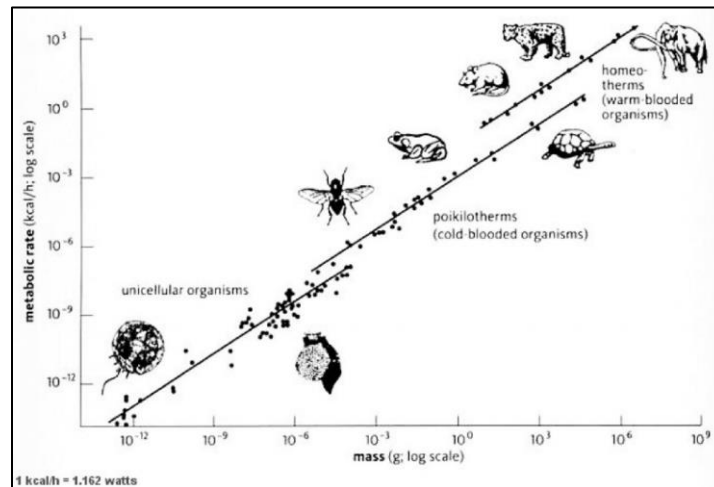
Surface Area: 14 in²
Volume: 3 in³
Ratio: 4.7



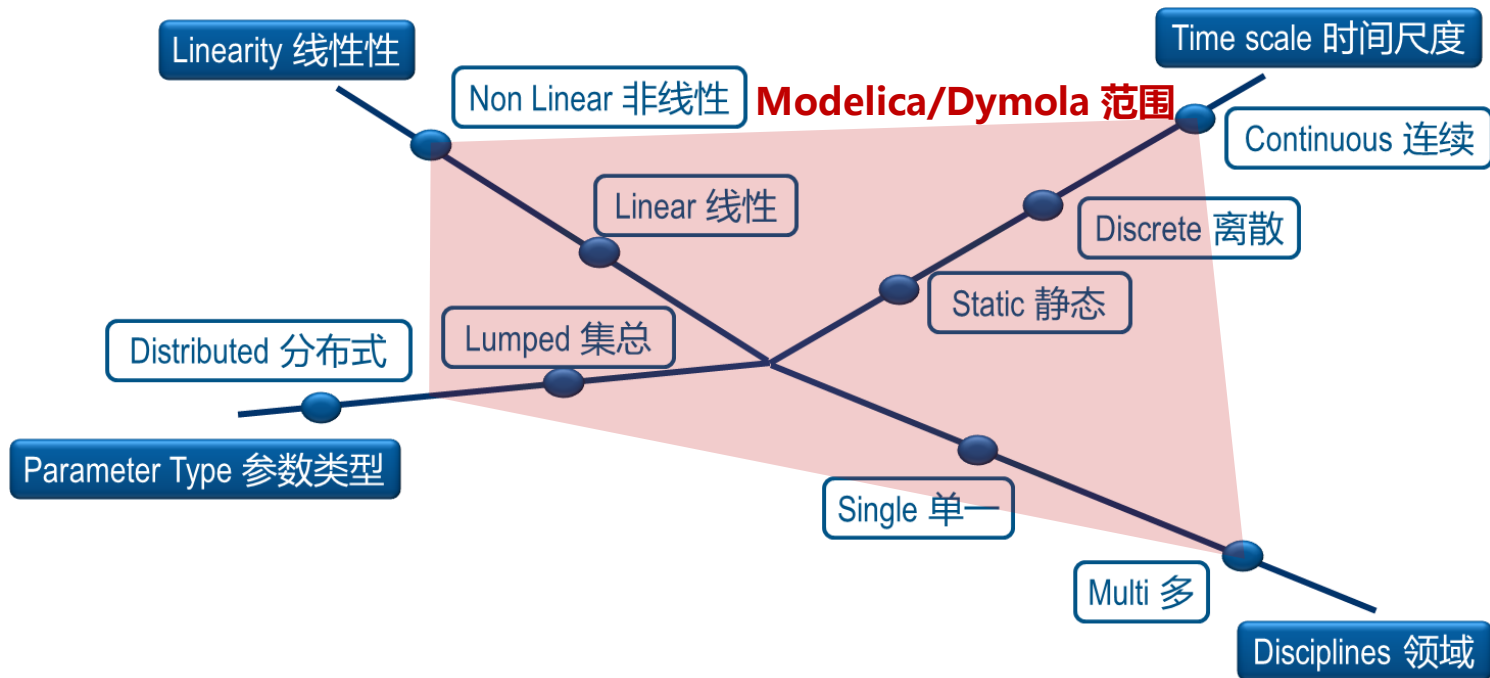
Elephant

Surface Area: 57,600 in²
Volume: 864,000 in³
Ratio: 0.067

“学科的”，“研究的”



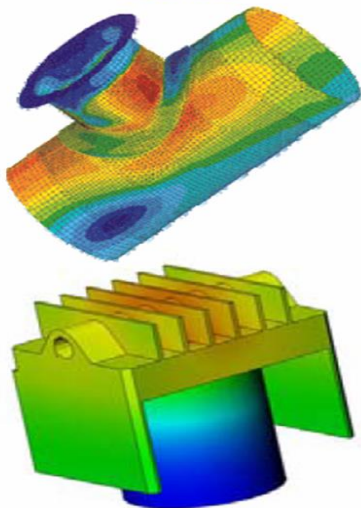
仿真模型形式(5/10)



仿真模型形式(6/10)

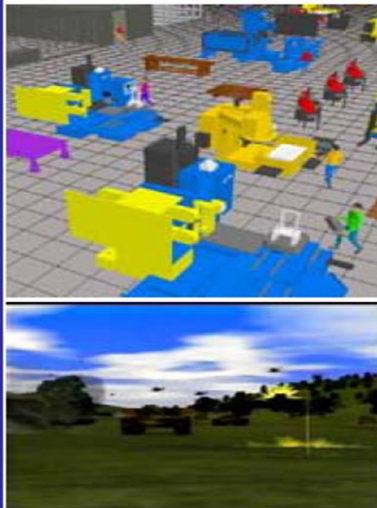
PDE

Static



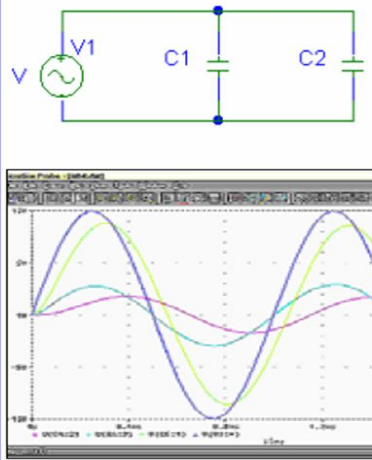
DEVS

Discrete Event



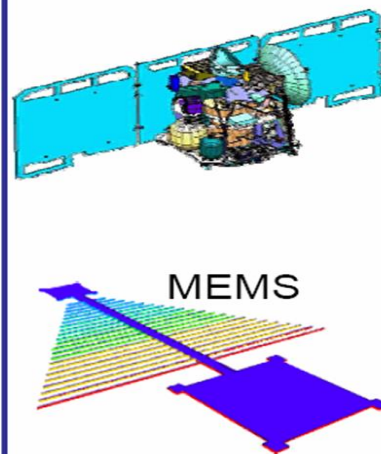
ODE
DAE
PDE

Continuous



Hybrid

Hybrid



仿真模型形式(7/10)

Algebraic Equations 代数方程

$$y + 9 = 15$$

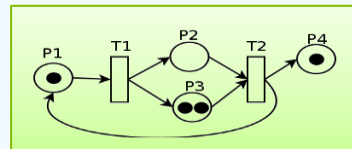
Ordinary Differential Equations (ODE) 常微分方程

$$\frac{d^2x}{dt} = -c \frac{dx}{dt} - x$$

Differential Algebraic Equations (DAE) 微分代数方程组

$$f\left(\frac{dx}{dt}, x, y, t\right) = 0$$

Discrete Event Systems Specification (DEVS) 离散系统规范



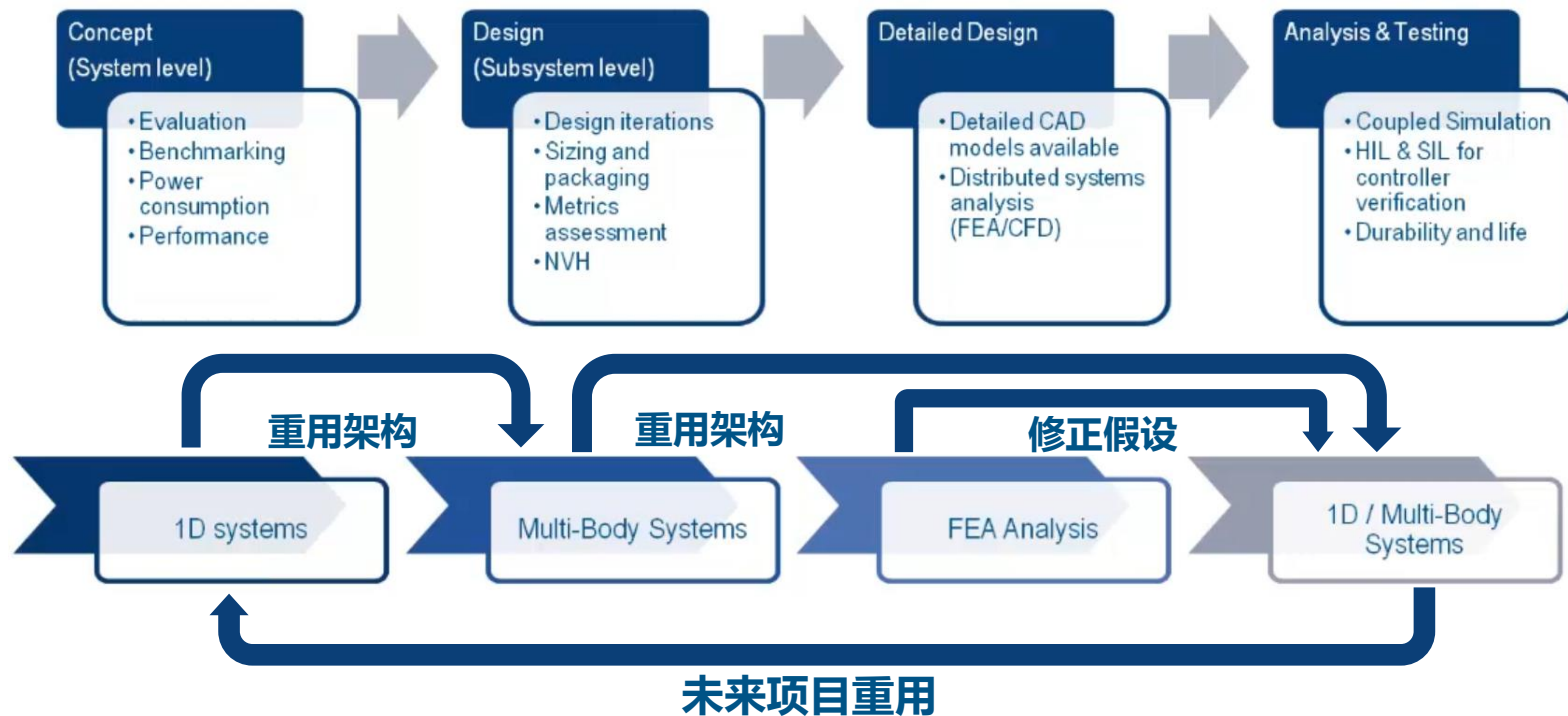
Hybrid Models 离散连续混合模型

Partial Differential Equations (PDE) 偏微分方程

$$i\hbar \frac{\partial}{\partial t} \psi = -\frac{\hbar^2}{2m} \left(\frac{\partial^2 \psi}{\partial x^2} + \frac{\partial^2 \psi}{\partial y^2} + \frac{\partial^2 \psi}{\partial z^2} \right)$$

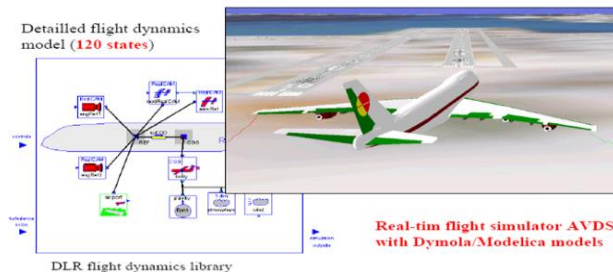
DYMOILA应用范围

仿真模型形式(8/10)

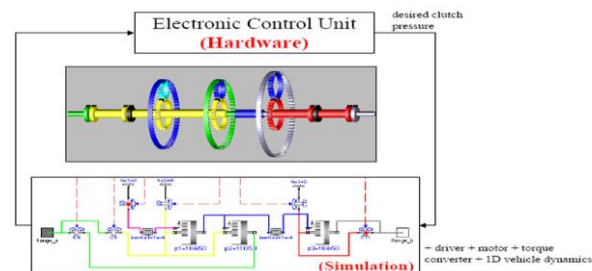


仿真模型范例(9/10)

Aircraft flight simulator AVDS with Modelica models (DLR)

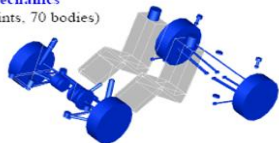


Hardware-in-the-loop simulation of automatic gearbox (different car manufacturers)



Large, detailed vehicle model (Ford Motors, Dynasim, DLR)

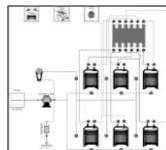
3D-Mechanics
(60 joints, 70 bodies)



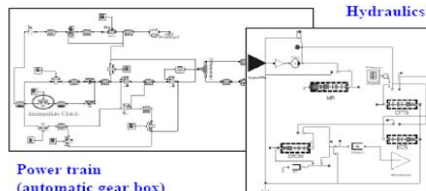
~ 25000 nontrivial algebraic equations

~ 320 states

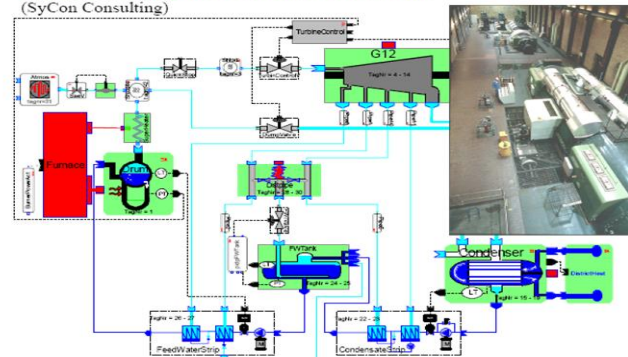
Motor (combustion)



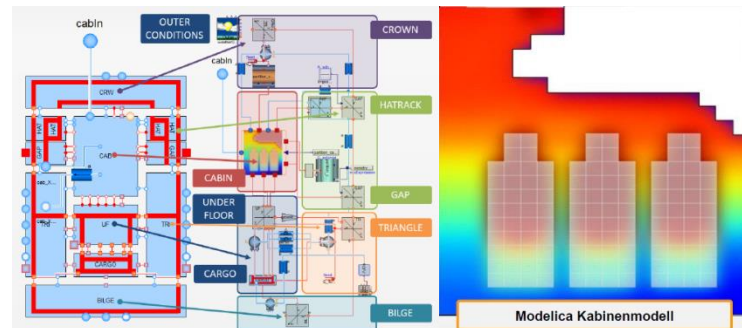
Hydraulics



Electric energy production and distribution (SyCon Consulting)



3DS.COM/CATIA © Dassault Systèmes | Confidential Information | 9/25/2015 | ref.: 3DS_Document_2015

[illegible]

培训内容

❖ 系统建模与仿真

- ❖ 什么是系统，为什么要仿真
- ❖ 仿真模型形式
- ❖ 仿真模型范例

❖ Modelica和Dymola概述

- ❖ 什么是Modelica语言
- ❖ 什么是Dymola软件
- ❖ 两者如何搭配工作
- ❖ 练习一：工业机器人

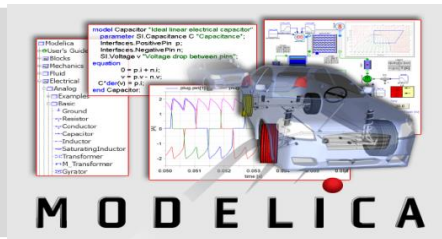
❖ Modelica语言初探

- ❖ Classes 类
- ❖ Types and Declaration 类型和声明
- ❖ Units 单位
- ❖ Inheritance 继承
- ❖ 练习二：单摆
- ❖ 练习三：月球登陆器
- ❖ Connectors 接口



Modelica 语言(1/2)

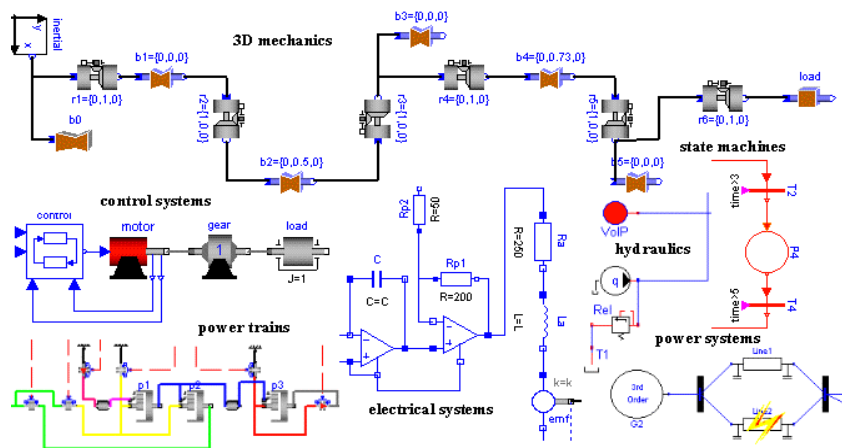
Modelica® 是一种开源，面向对象，基于方程的系统建模语言。
描述对象包括 机械，电气，电子，液压，热传导，控制，电力驱动，工厂过程等等。



$$\begin{cases} \dot{x} = f(x, u, t) \\ y = h(x, u, t) \end{cases}$$

$$\begin{cases} \dot{x} = A x + B u \\ y = C x + D u \end{cases}$$

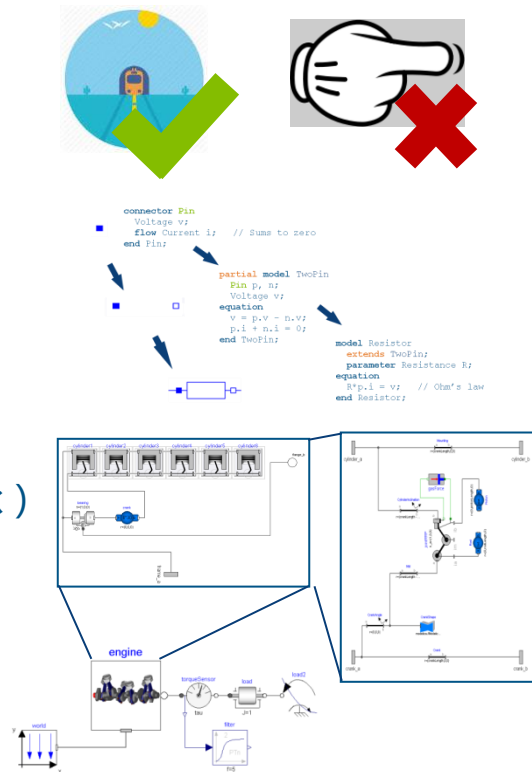
```
model SimplePendulum
  Real theta;
  Real omega(start=1);
  parameter Real g = 9.81;
  parameter Real L = 1;
equation
  der(omega) = -(g/L)*sin(theta);
  der(theta) = omega;
end SimplePendulum;
```



Modelica 语言(2/2)

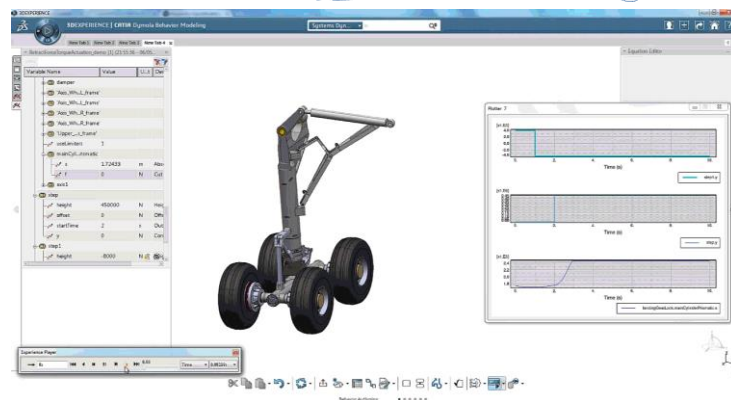
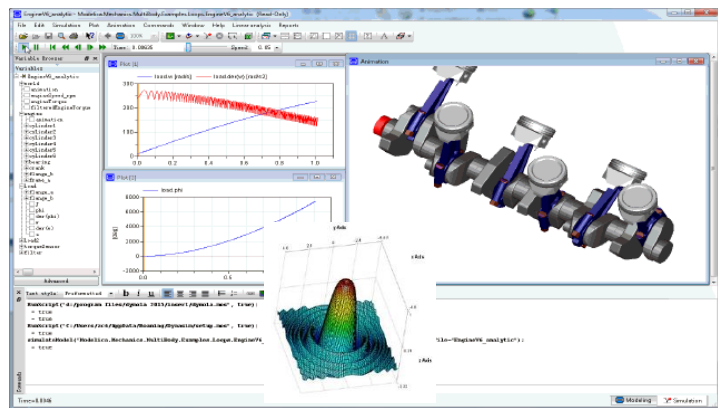
Modelica 语言特性:

- ❖ Modelica是一种描述性的**建模语言**而不是指令性的编程语言
- ❖ 所以允许**非因果性**建模
- ❖ 建模利用的是与物理定律（而不是算法）一致的**声明性方程**
 - ❖ $1/4 * (\text{der}(x) + 3) = x^2 - a$; 完全等价于 $\text{der}(x) = 4 * (x^2 - a) - 3$;
- ❖ Modelica模型可以是微分方程，代数方程和离散方程
 - ❖ $\text{der}(x) = \exp(x) * x$; $x + a = b$; $u = 1 + \text{pre}(u)$;
- ❖ **基于对象**建模语言特性（class 类，instance 实例，inheritance 继承）
 - ❖ Resistor.PositivePin.Voltage
- ❖ **层级化**建模
- ❖ 由国际Modelica协会发展和维护的**开源语言**



Dymola软件(1/2)

Dymola (Dynamic Modeling Lab 动态建模实验室), 是基于公开Modelica系统建模语言的集成建模和仿真环境



- ❖ 具有丰富多领域模型库或可利用简单易懂的Modelica语言开发专属部件
- ❖ 图形化界面对模型进行参数配置或与PLM参数对象关联
- ❖ 通过连接部件快速建立多层次复杂系统模型
- ❖ 强大的仿真引擎以及开箱即用的后处理，可视化和3维展示
- ❖ 独有的方程符号处理器及数字求解器并可输出高质量代码用于基于模型预测控制设计

Dymola 软件(2/2)

Dymola的开放性:

- ❖ Dymola允许用户自行用Modelica语言描述物理方程构建属于自己的模型
- ❖ Dymola环境完全开放，即用户除了可以构建新部件以外，还采用延展和复制的方式
- ❖ Modelica标准库及各种商业库中大部分元件代码都对用户开放

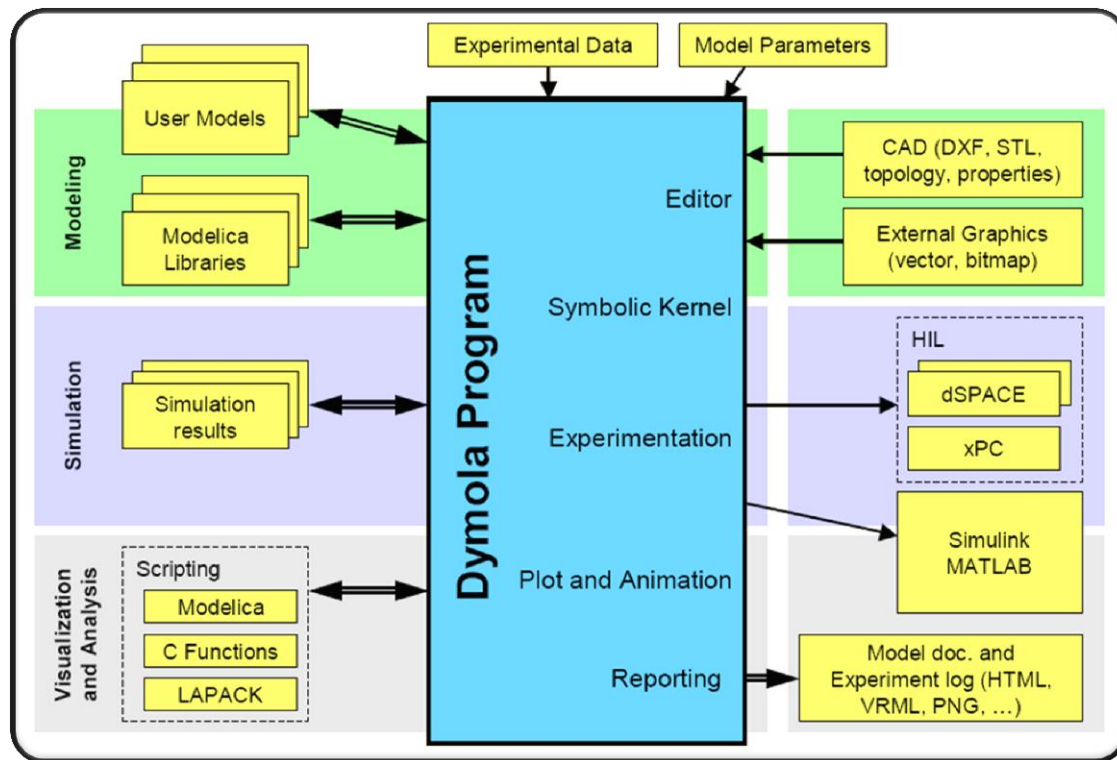
```
model Resistor "Ideal linear electrical resistor"
  parameter Modelica.SIunits.Resistance R(start=1)
    "Resistance R_ref at temperature T_ref";
  parameter Modelica.SIunits.Temperature T_ref=300.15 "Reference temperature";
  parameter Modelica.SIunits.LinearTemperatureCoefficient alpha=0
    "Temperature coefficient of resistance (R_actual = R_ref*(1 + alpha*(heatPort.T - T_ref)))";

  extends Modelica.Electrical.Analog.Interfaces.OnePort;
  extends Modelica.Electrical.Analog.Interfaces.ConditionalHeatPort(T = T_ref);
  Modelica.SIunits.Resistance R_actual
    "Resistance = R_ref*(1 + alpha*(internalHeatPort.T - T_ref))";
  B;
equation
  assert((1 + alpha*(T_heatPort - T_ref)) >= Modelica.Constants.eps, "Temperature outside scope of model!");
  R_actual = R*(1 + alpha*(T_heatPort - T_ref));
  v = R_actual*i;
  LossPower = v*i;
end Resistor;
```

3DS.COM/CATIA © Dassault Systèmes | Confidential Information | 9/25/2015 | ref.: 3DS_Document_2015

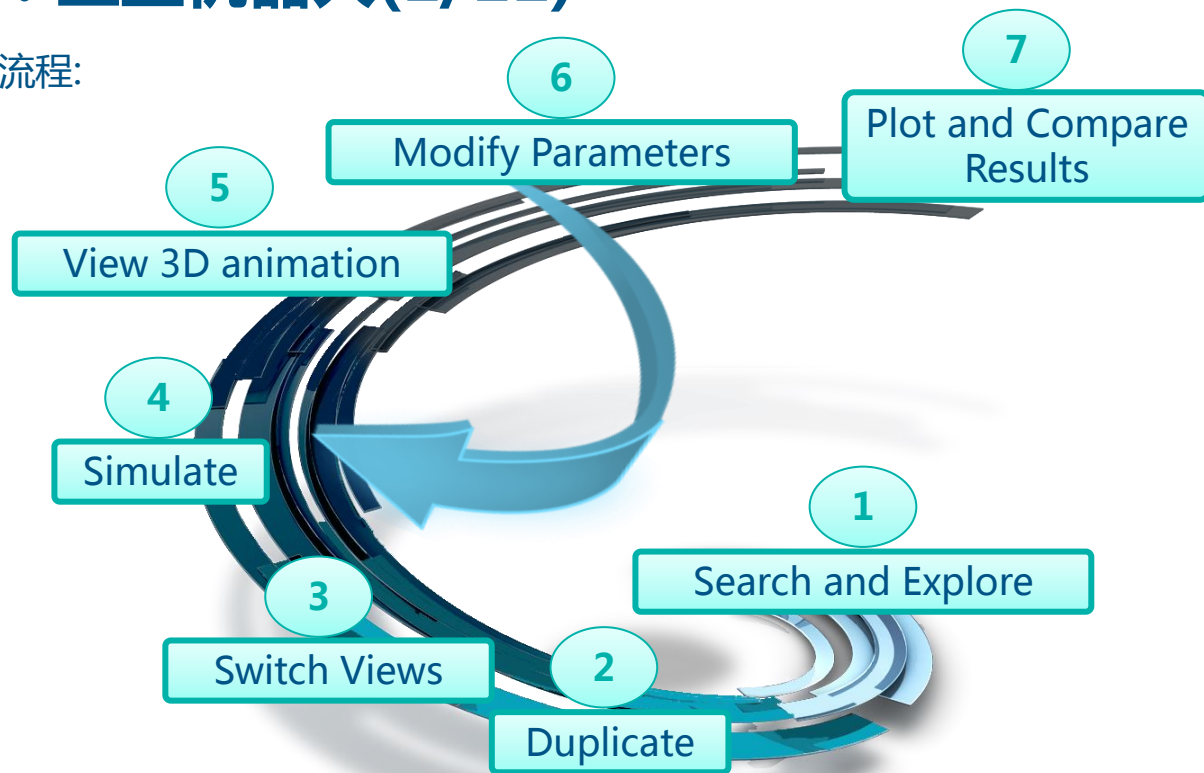


DYMOLA搭配Modelica(2/2)



练习一：工业机器人(1/11)

此练习基本流程:



练习一：工业机器人(2/11)

以下几种方式可找到机器人范例:

- ❖ From File -> Demos -> Robot menu

OR:

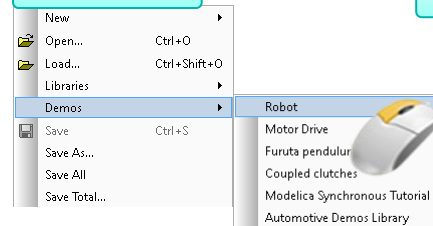
- ❖ Exploring the Modelica Standard Library: the path is Modelica.Mechanics.MultiBody.Examples.Systems.RobotR3.fullRobot

OR:

- ❖ Using the search option from the contextual menu

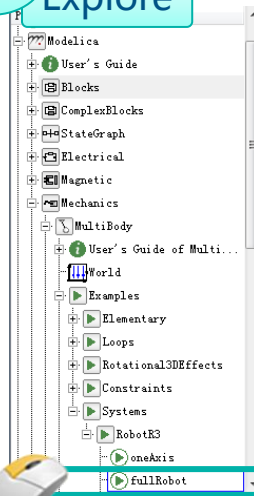
1

Menu



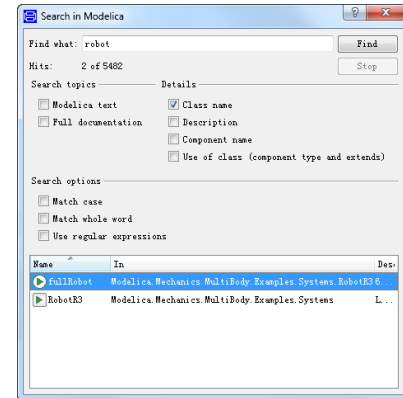
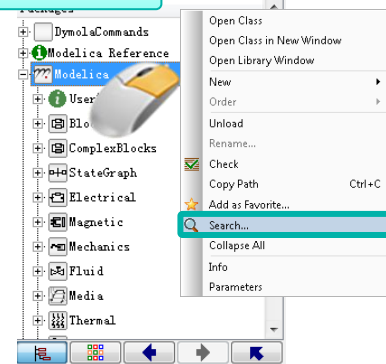
1

Explore



1

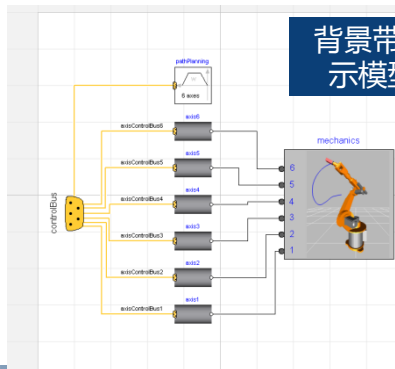
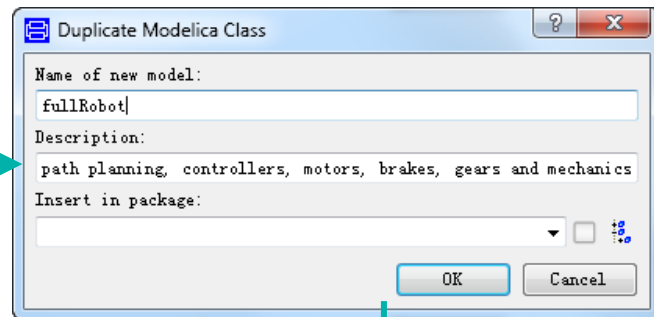
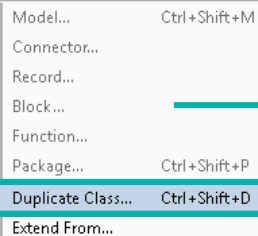
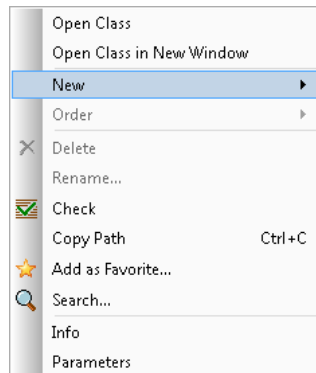
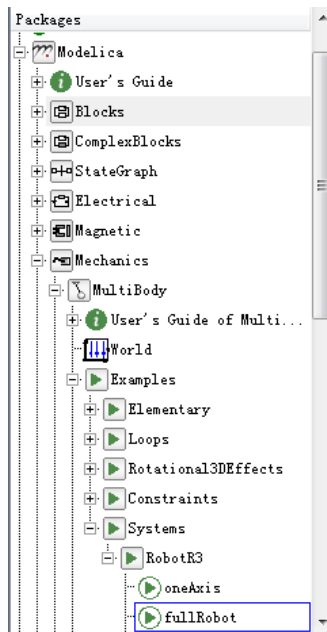
Search



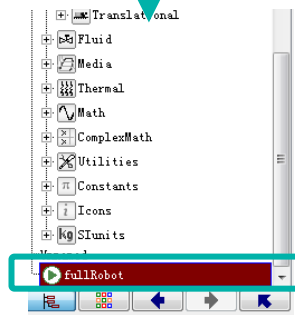
练习一：工业机器人(3/11)

2

Duplicate



背景带有网格表示模型可编辑



练习一：工业机器人(4/11)

3

Switch Views



模型库

Tree view showing the library structure with categories like Mechanical, Electrical, and Magnetic.

图标界面

Icon view showing a 3D model of a robotic arm with a coordinate system and a list of components.

Modelica代码界面

```
parameter Boolean animation=true "true, if animation shall be enabled";
parameter SI.Mass load(m)=15 "Mass of load";
parameter SI.Position loadPos=(0,0,25,0) "Distance from last flange to load mass";
parameter SI.Acceleration g=9.81 "Gravity acceleration";
SI.Angle q[6] "Joint angles";
SI.AngularVelocity qd[6] "Joint speeds";
SI.AngularAcceleration qdd[6] "Joint accelerations";
SI.Torque tau[6] "Joint driving torques";
//q0=(0,0,351,0);

equation
q = {x1.phi,x2.phi,x3.phi,x4.phi,x5.phi,x6.phi};
qd = der(q);
qdd = der(qd);
tau = {x1.tau,x2.tau,x3.tau,x4.tau,x5.tau,x6.tau,x7.tau};
end MechanicalStructure;
```

当前模型

Diagram view showing the mechanical components of the robot (multibody system) connected by joints and actuators.

说明文档界面

Parameters table:

Type	Name	Default	Description
Boolean	animation	true	true, if animation shall be enabled
Mass	load	15	Mass of load [kg]
Position	loadPos	(0,0,25,0)	Distance from last flange to load mass [m]
Acceleration	g	9.81	Gravity acceleration [m/s^2]

Connectors table:

Type	Name	Description
Flange	x1	
Flange	x2	
Flange	x3	
Flange	x4	
Flange	x5	
Flange	x6	

部件参数界面

Parameters table:

Type	Name	Default	Description
Boolean	animation	true	true, if animation shall be enabled
Mass	load	15	Mass of load [kg]
Position	loadPos	(0,0,25,0)	Distance from last flange to load mass [m]
Acceleration	g	9.81	Gravity acceleration [m/s^2]

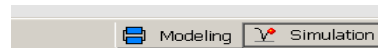
Connectors table:

Type	Name	Description
Flange	x1	
Flange	x2	
Flange	x3	
Flange	x4	
Flange	x5	
Flange	x6	

练习一：工业机器人(5/11)

页面右下角从建模界面切换到仿真界面(Simulation)

仿真界面下可通过仿真工具条进行常用仿真设置



Run Script

Translate

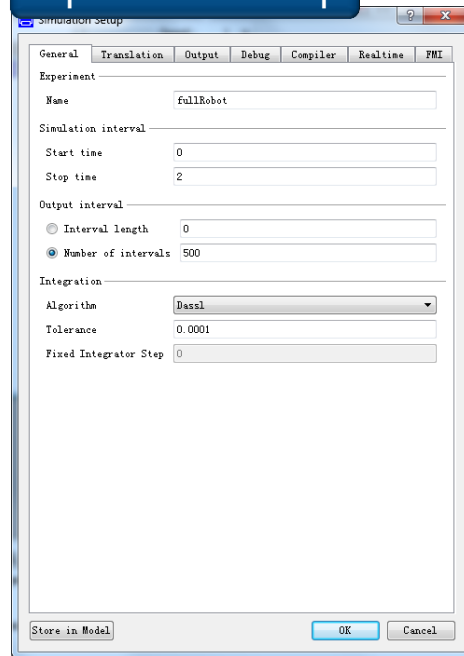
将当前模型从Modelica代码翻译成计算机可执行代码（C或DLL或FMU），翻译后可进行仿真或代码输出

Simulate

根据仿真实验设置对一段仿真周期执行仿真。若Modelica模型之前没有翻译，这里将会自动先翻译后仿真

Stop

Experiment Setup



练习一：工业机器人(6/11)

Simulation interval group:

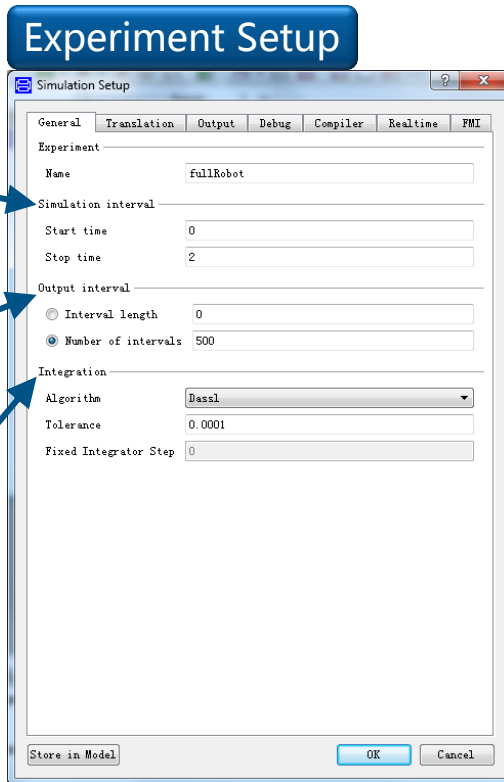
设定开始及结束时间

Output interval group:

指定仿真结果以何种时间间隔进行存储。可以直接设定存储间隔时间（步长）或设定仿真时间内存储步数。请注意这里的步长不是仿真计算步长而是存储步长以用于结果显示或后处理。

Integration group:

根据仿真模型的系统特性需要可指定仿真的数值积分算法和容差度



练习一：工业机器人(7/11)

点击翻译然后（或直接）进行仿真



4

Simulate

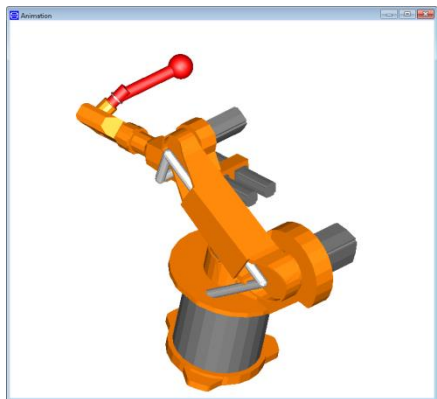


在菜单栏里的**Simulation > Show Log**，可以看到模型在翻译和仿真中的具体执行信息

如果动画窗口在仿真后没有自动出现，可以点击**Animation > New Animation Window**

5

View 3D animation



Animation toolbar 动画工具条

View controls

Operation	Meta Key	Mouse	Arrow key
Moving up/down/left/right	None	Up/Down/Left/Right	Up/Down/Left/Right
Tilt (rotate around x-axis)	Ctrl	Up/Down	Up/Down
Pan (rotate around y-axis)	Ctrl	Left/Right	Left/Right
Roll (rotate around z-axis)	Ctrl+Shift	Clockwise/Counterclockwise	Left/Right
Zoom in/out	Shift	Up/Down	Up/Down
Zoom in/out	None	Wheel	None

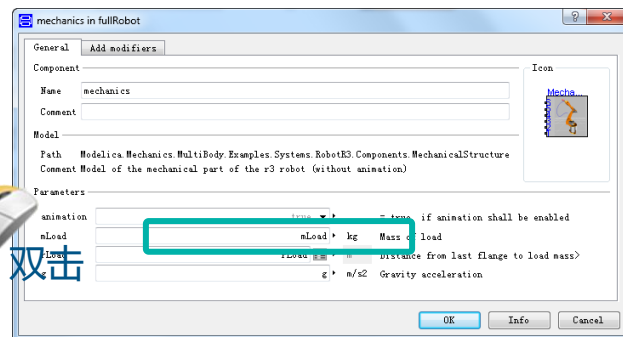
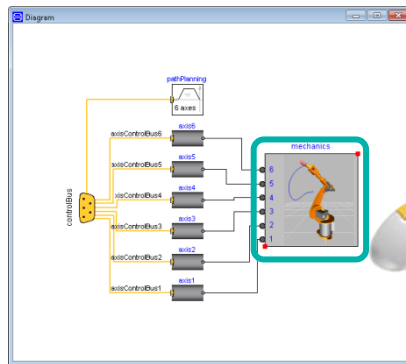


练习一：工业机器人(8/11)

6

Modify Parameters

通过两种方式更改模型参数：在参数列表中更改或打开模型工具条找到相应部件参数



双击

将机器人负载质量更改为30kg后再执行一次仿真

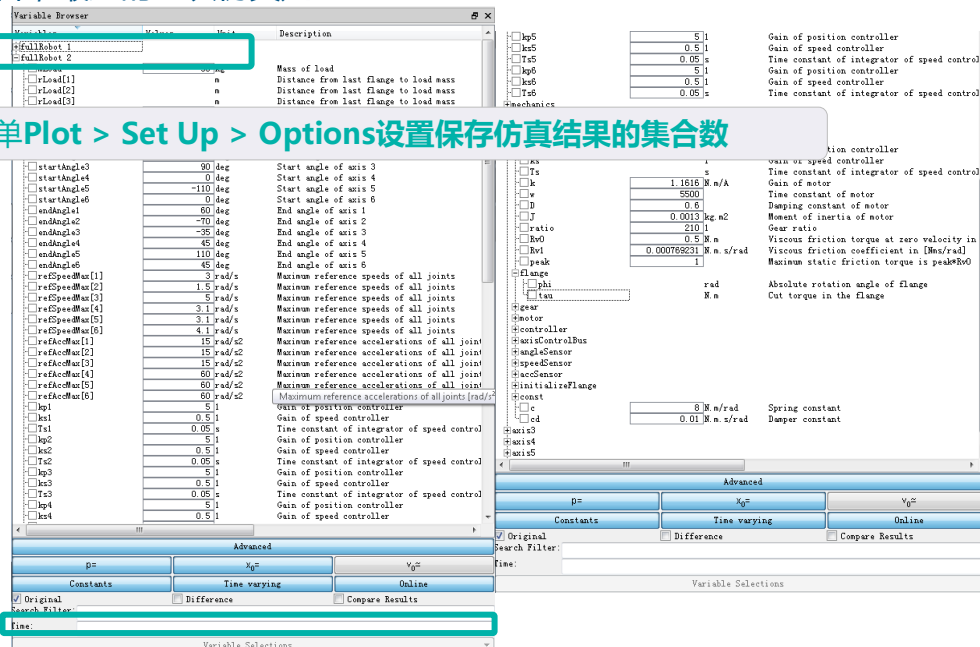


练习一：工业机器人(9/11)

The variable browser 显示了所有参数和变量的树状结构图。可以看到参数和变量都是以子系统（子模型）层级化的进行组织
最高层级节点是不同仿真结果的集合，最近的一次仿真是fullRobot 2

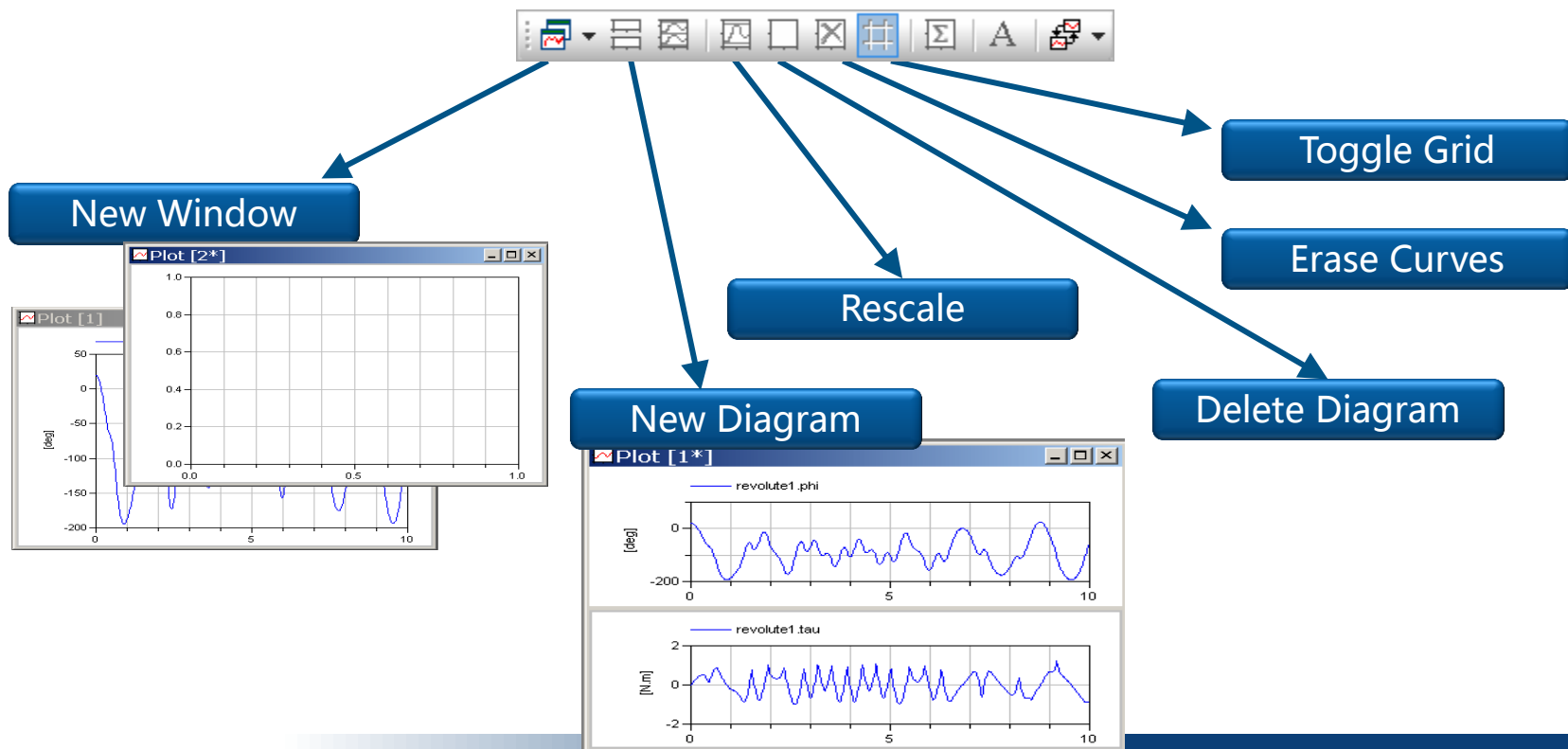


可以在菜单Plot > Set Up > Options设置保存仿真结果的集合数



对于变量，可以通过输入时间可以看到指定时间下变量的值

练习一：工业机器人(10/11)

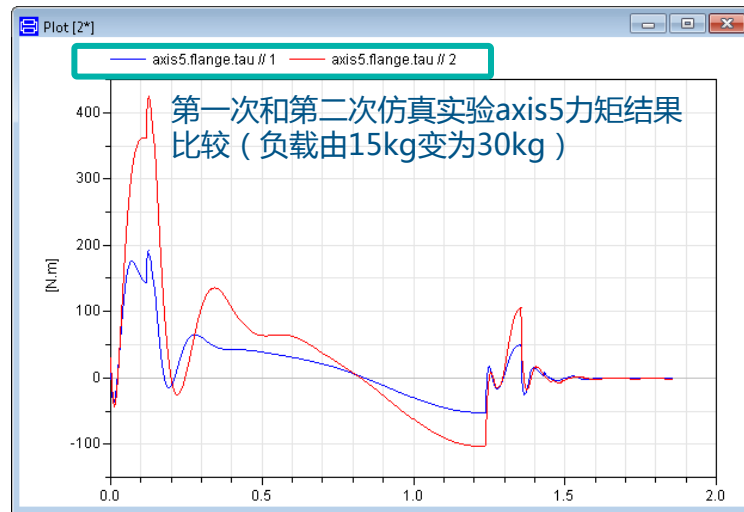
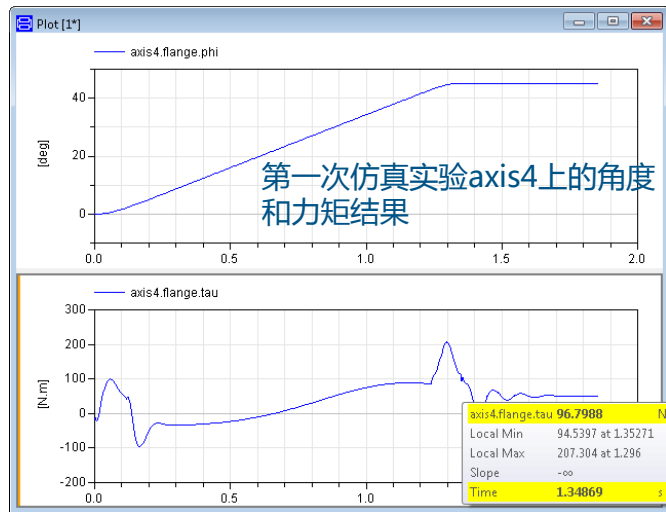


练习一：工业机器人(11/11)

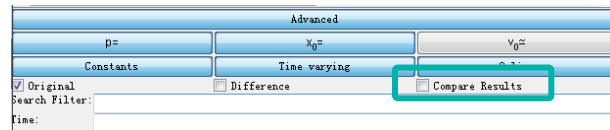
Plot the results of your simulations by selecting a plot window and checking the desired variables from the variable browser tree. Use the plotting tools to get the windows below.

7

Plot and Compare Results



比较结果时，除了可以将需要显示的结果逐个点选以外，还可以激活 Advanced->Compare Results，所有仿真试验下同一名称变量会自动同时选中。例上只要选取axis5力矩，所有实验结果里的axis5力矩会自动显示



培训内容

◆ 系统建模与仿真

- 什么是系统，为什么要仿真
- 仿真模型形式
- 仿真模型范例

Modelica和Dymola概述

- ❖ 什么是Modelica语言
- ❖ 什么是Dymola软件
- ❖ 两者如何搭配工作
- ❖ 练习一：工业机器人

❖ Modelica语言初探

- ❖ Classes 类
- ❖ Types and Declaration 类型和声明
- ❖ Units 单位
- ❖ Inheritance 继承
- ❖ 练习二：单摆
- ❖ 练习三：月球登陆器
- ❖ Connectors 接口

❖ Modelica语言初探（续）

- ❖ 练习四：雨刮器
- ❖ Acausality 非因果性
- ❖ 练习五：直流电机
- ❖ Discrete Events 离散事件
- ❖ Hybrid Modeling 混合建模
- ❖ 练习六：受控直流电机
- ❖ 练习七：温度控制
- ❖ Arrays 数组
- ❖ 练习八：振动梁和热传导

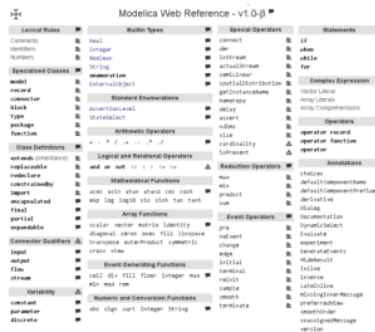
 Dymola仿真环境

- ❖ 建模窗口
- ❖ 文本编辑窗口
- ❖ 浏览及生成文档报告
- ❖ 仿真窗口
- ❖ 基础仿真脚本
- ❖ 练习九：RV-2AJ 机器人



Modelica® - A Unified Object-Oriented Language for Systems Modeling

Language Specification

Version 3.3

Classes 类(1/3)

Modelica class类的基本结构

```
class ModelName
```

```
    //公开参数、变量或输入/输出声明，  
    // 可被模型外部以“模型名称.参数（变量）名称”的方式访问获取  
    TypeName VariableName;  
    ...
```

```
protected
```

```
    // 内部（非公开）参数或变量声明  
    TypeName VariableName;  
    ...
```

```
equation
```

```
    // algebraic and differential equations 代数方程和微分方程  
    Expression = Expression;  
    ...
```

```
end ModelName
```

Classes 类(2/3)

Modelica各种类

Model: 物理（受控）部件的类

```
model Ground "Ground node"

    Interfaces.Pin p a;
    a;
equation
    p.v = 0;
end Ground;
```

Connector: 不同Model类交互信息的接口

```
connector PositivePin "Positive pin of an electric component"
    SI.Voltage v "Potential at the pin";
    flow SI.Current i "Current flowing into the pin";
    a;
end PositivePin;
```

Record : 只包含参数和变量声明，不包含equation方程

Model...	Ctrl+Shift+M
Connector...	
Record...	
Block...	
Function...	
Package...	Ctrl+Shift+P

Classes 类(3/3)

Modelica各种类

Block: 固定因果性的类，用于非物理部件（控制或以算法描述的物理部件）

```
block Feedback
  "Output difference between commanded and feedback input"

  input Interfaces.RealInput u1 a;
  input Interfaces.RealInput u2 a;
  output Interfaces.RealOutput y a;
  a;

equation
  y = u1 - u2;
end Feedback;
```

Function: 用于定义算法的类

```
function length
  "Return length of a vectorReturn length of a vector (better as:
  extends Modelica.Icons.Function;
  input Real v[:] "Vector";
  output Real result "Length of vector v";
  a;
algorithm
  result := sqrt(v*v);
end length;
```

Package: 用于组织Modelica library库结构的类，可包含上述的各种Modelica类，包括本身。

Model...	Ctrl+Shift+M
Connector...	
Record...	
Block...	
Function...	
Package...	Ctrl+Shift+P

Types and Declaration 类型和声明(1/2)

Type Specifiers , Prefixes and Attributes 类型区分符 , 前缀和特性

Type Specifiers	Comments
Real	Real type 实型
Integer	Integer type 整型
Boolean	Boolean Type (true or false) 布尔型

Type Prefixes	Comments
parameter	参数, 单次仿真中固定不变值, 但在仿真初始可进行声明改变
constant	常数, 单次仿真中固定不变值, 仿真初始时也不可进行声明改变
discrete	离散型常数

Type Attributes	Comments
quantity	定义数量
unit	定义单位
min	设定的仿真过程中变量的最小值
max	设定的仿真过程中变量的最大值
fixed	= true: Value of "start" is a fixed initial condition 变量初始值采用由人为设定的值 = false : Value of "start" is a "guess" value 变量初始值采用计算机“猜测”的值
nominal	数值算法进行scaling的名义值

Types and Declaration 类型和声明(2/2)

Type Specifiers , Prefixes and Attributes 类型区分符 , 前缀和特性



可根据自身需要定义Type

```
type Real // Note: Defined with Modelica syntax although predefined
RealType value; // Accessed without dot-notation
parameter StringType quantity = "";
parameter StringType unit = "" "Unit used in equations";
parameter StringType displayUnit = "" "Default display unit";
parameter RealType min=-Inf, max=+Inf; // Inf denotes a large value
parameter RealType start = 0; // Initial value
parameter BooleanType fixed = true, // default for parameter/constant;
                                = false; // default for other variables
parameter RealType nominal; // Nominal value
parameter StateSelect stateSelect = StateSelect.default;
equation
assert(value >= min and value <= max, "Variable value out of limit");
assert(nominal >= min and nominal <= max, "Nominal value out of limit");
end Real;
```

一个上限是15米，初始值是3米的长度变量

```
Real myLength( start=3, max=15, unit="m", quantity="Length" );
```

Units 单位(1/1)

Modelica Standard Library(MSL) 已经预定义了覆盖多个学科领域的单位



关于MSL后面会有详细介绍。
也可根据自身需要定义Units

- ❖ Space and Time, Periodic and Related Phenomena
- ❖ Mechanics, Acoustics, Heat
- ❖ Electricity and Magnetism, Light and Related Electro-Magnetic Radiations
- ❖ Physical Chemistry, Atomic and Nuclear Physics
- ❖ Nuclear Reactions and Ionizing Radiations
- ❖ Characteristic Numbers
- ❖ Solid State Physics

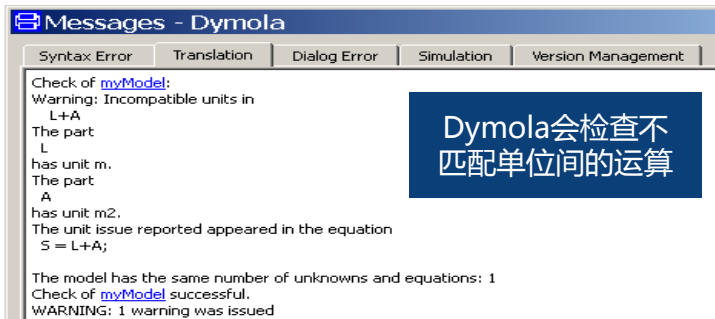
```
Real myLength( start=3, max=15, unit="m", quantity="Length");
```

所以上述类型定义例子也可写成：

```
Modelica.SIunits.Length myLength( start=3, max=15);
```

```
model myModel
parameter Modelica.SIunits.Length L;
parameter Modelica.SIunits.Area A;
Modelica.SIunits.Length S;
equation |
S=L+A;
end myModel;
```

Check

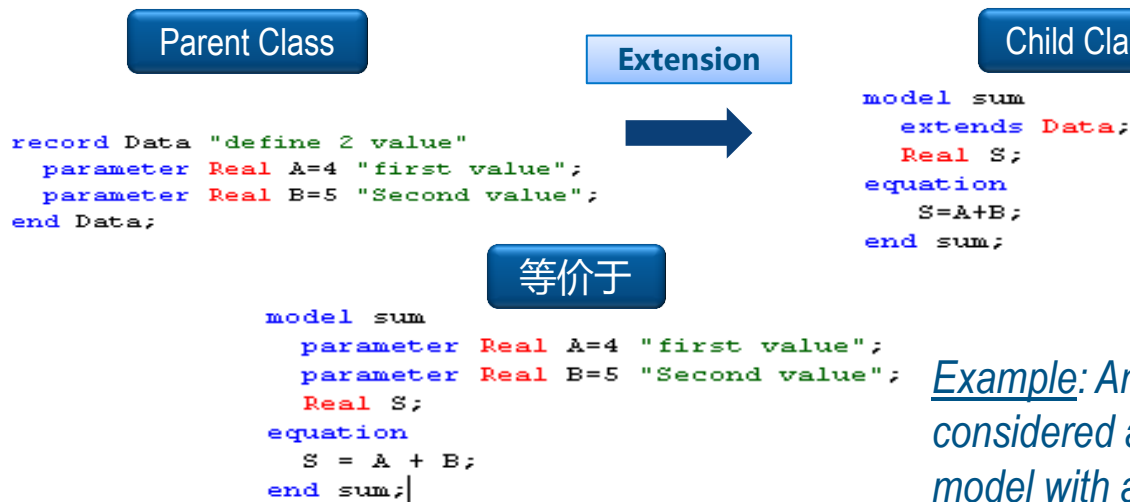


Dymola会检查不
匹配单位间的运算

Inheritance 继承(1/1)

基于对象(OO, Object-Oriented)的建模方式其中一个优势是可基于已有类进行行为和属性的延展(extend)构成新的类。

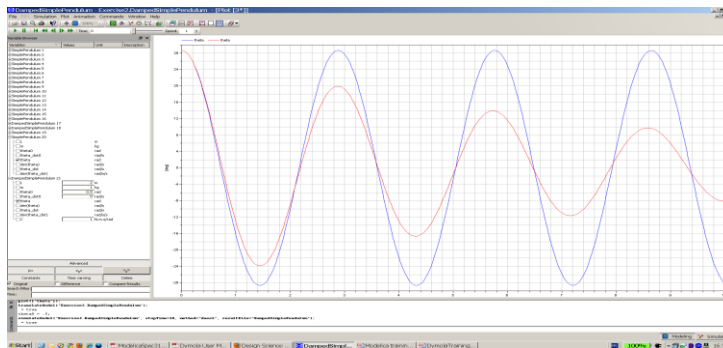
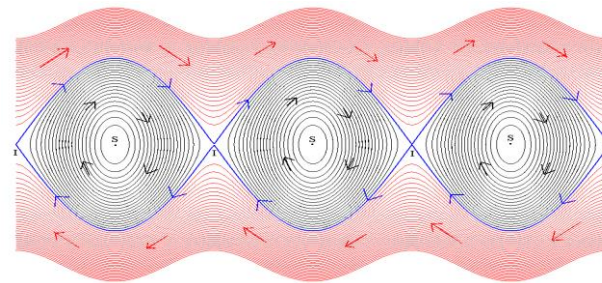
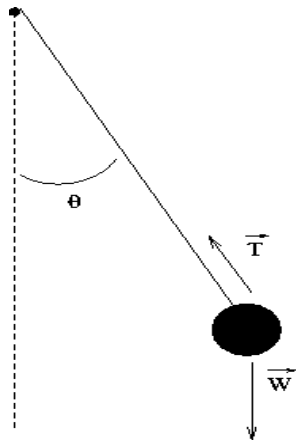
已有的类在延展的过程中称为基类(base class)或父类(parent class), 通过延展得到的类叫做derived class或subclass或child class



Example: An heating resistor model can be considered as an extension of a simple resistor model with an extra heat related equation...

练习二：单摆(1/10)

$$mL^2 \ddot{\theta} + c\dot{\theta} + mgL \sin(\theta) = 0$$



练习二：单摆(2/10)

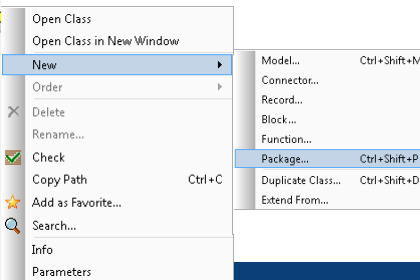
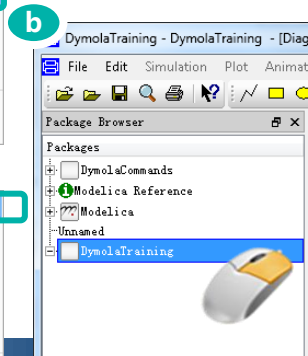
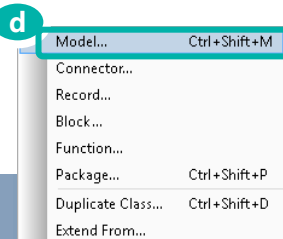
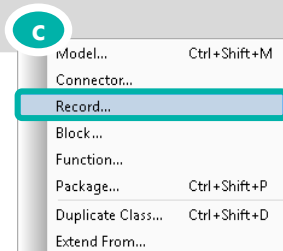
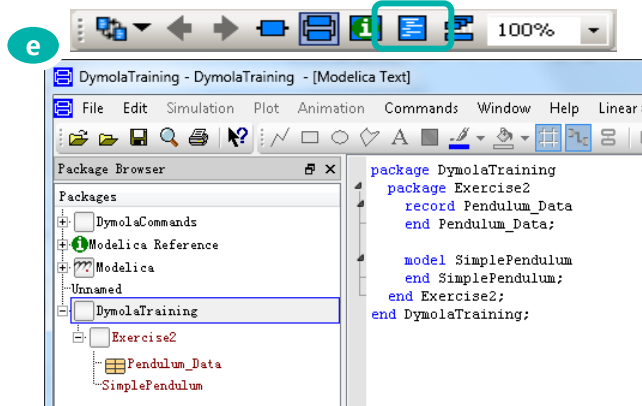
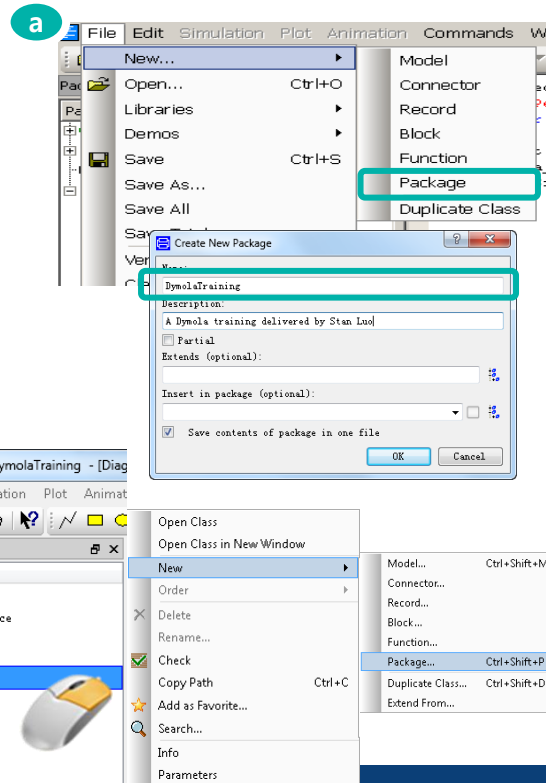
此练习基本流程：



练习二：单摆(3/10)

创建Package, Record 和 Model

- 创建顶层Package: **File > New... > Package** 命名 **DymolaTraining**
- 在 **DymolaTraining** 里创建新 Package 命名 **Exercise2**
- 在 **Exercise2** 里创建新 Record 命名 **Pendulum_Data**
- 在 **Exercise2** 里创建新 Model 命名 **SimplePendulum**
- 点击代码界面查看结果

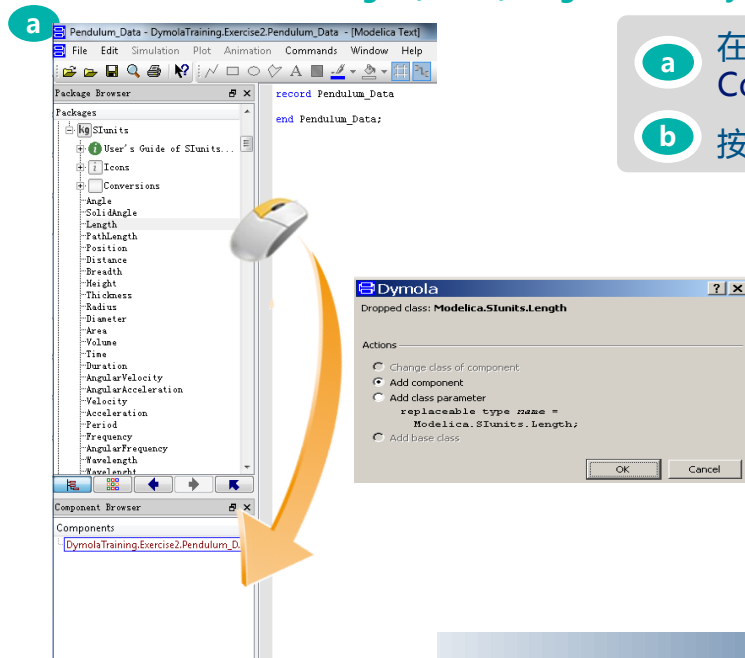


练习二：单摆(4/10)

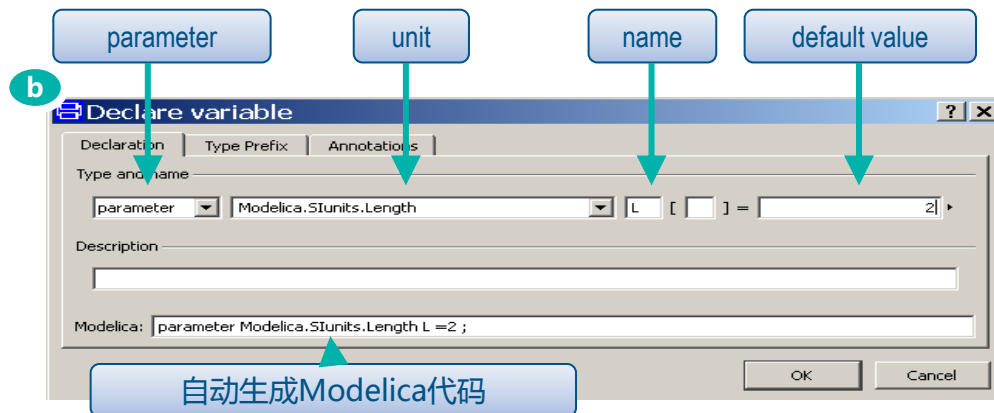
$$mL^2\ddot{\theta} + c\dot{\theta} + mgL\sin(\theta) = 0$$

Record 类里包含单摆模型的所有参数和变量:

- ❖ 参数Parameters: **Mass(m)**, **length(L)**, **damping constant(c)**, **initial angle(theta_0)**, **initial angular velocity(theta_dot0)**
- ❖ 变量Variables: **Angle (theta)**, **angular velocity(theta_dot)**



- 在Modelica标准库里找到 **SIunits.Length** 并拖拽至 **Pendulum_Data**的 Component Browser下 (操作前确认当前激活模型是Pendulum_Data)
- 按图提示在对话框输入长度参数 **L** 的信息



自动生成Modelica代码

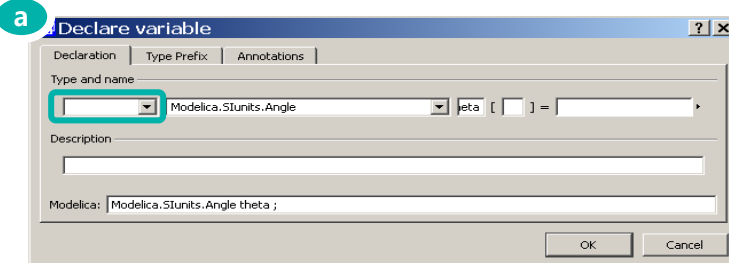
练习二：单摆(5/10)

$$mL^2\ddot{\theta} + c\dot{\theta} + mgL \sin(\theta) = 0$$

声明参数和变量

a 对于变量，如 **theta**，前缀第一列信息留空

b 如图将所有模型参数/变量输入完整



也可以在Siunits里进行搜索然后将结果直接拖拽到Pendulum_Data的component browser中

b

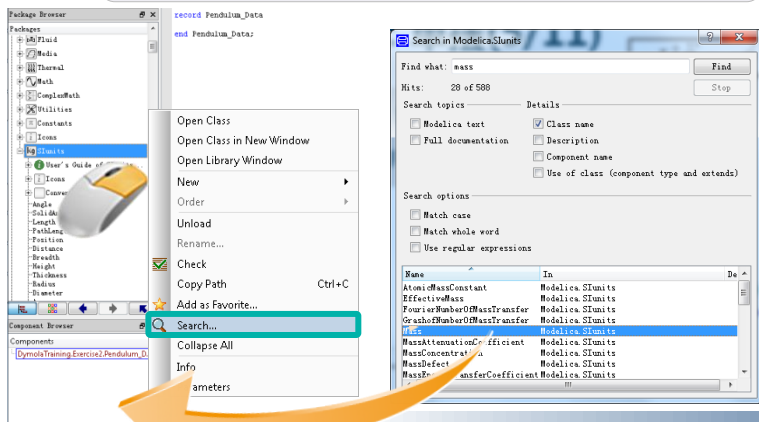
```
record Pendulum_Data

//parameter declaration
parameter Modelica.SIunits.Length L=2;
parameter Modelica.SIunits.Mass m=1;
parameter Modelica.SIunits.RotationalDampingConstant C=1;

parameter Modelica.SIunits.Angle theta0=0;
parameter Modelica.SIunits.AngularVelocity theta_dot0=0;

//variable declaration
Modelica.SIunits.Angle theta(start=theta0);
Modelica.SIunits.AngularVelocity theta_dot(start=theta_dot0);

end Pendulum_Data;
```



练习二：单摆(6/10)

$$mL^2\ddot{\theta} + c\dot{\theta} + mgL \sin(\theta) = 0$$

- a** 也可以直接在代码界面直接进行参数/变量声明，用图中**替代路径(Alias)**的方式可以提高效率
- b** 如果关键字没有颜色高亮显示，可在代码空白区域激活**右键菜单**进行设置或使用快捷键

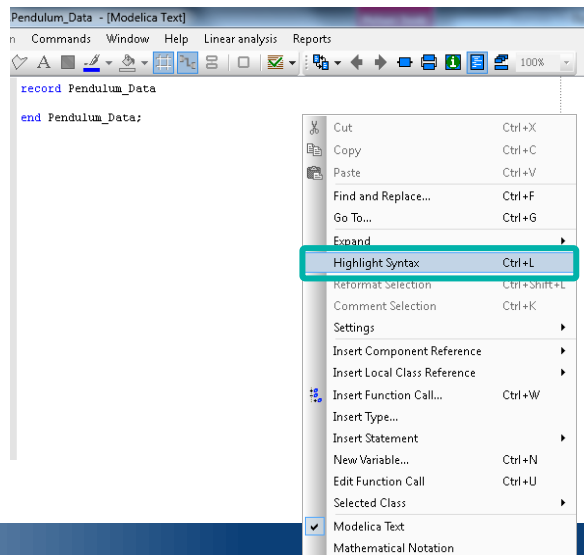
a

```
import SI = Modelica.SIunits;

parameter SI.Length l=2;
parameter SI.Mass m=1;
parameter SI.RotationalDampingConstant C=1;

parameter SI.Angle theta0=0;
parameter SI.AngularVelocity theta_dot0=0;

//variable declaration
SI.Angle theta(start=theta0);
SI.AngularVelocity theta_dot(start=theta_dot0);
```



练习二：单摆(7/10)

SimplePendulum 模型将基于 Pendulum_Data 进行延展并加入单摆运动方程

$$mL^2\ddot{\theta} + c\dot{\theta} + mgL\sin(\theta) = 0$$

```
model SimplePendulum
  extends Pendulum_Data;

  equation
    theta_dot = der(theta);
    der(theta_dot) = -Modelica.Constants.g_n/L*sin(theta) - c/(m*L^2)*theta_dot;
  end SimplePendulum;
```

3

利用继承

Modelica 重力常量

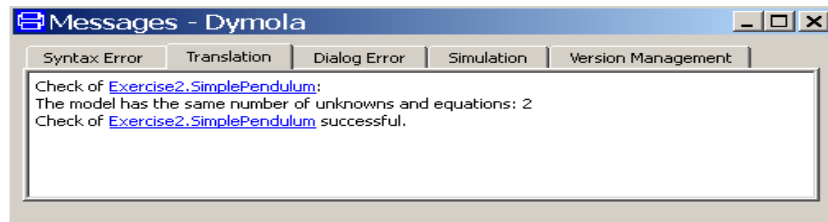
4

输入公式



Modelica语言目前不支持多级求导嵌套，即der(der(theta))，因此在此例子中需要构造角加速度theta_dot变量

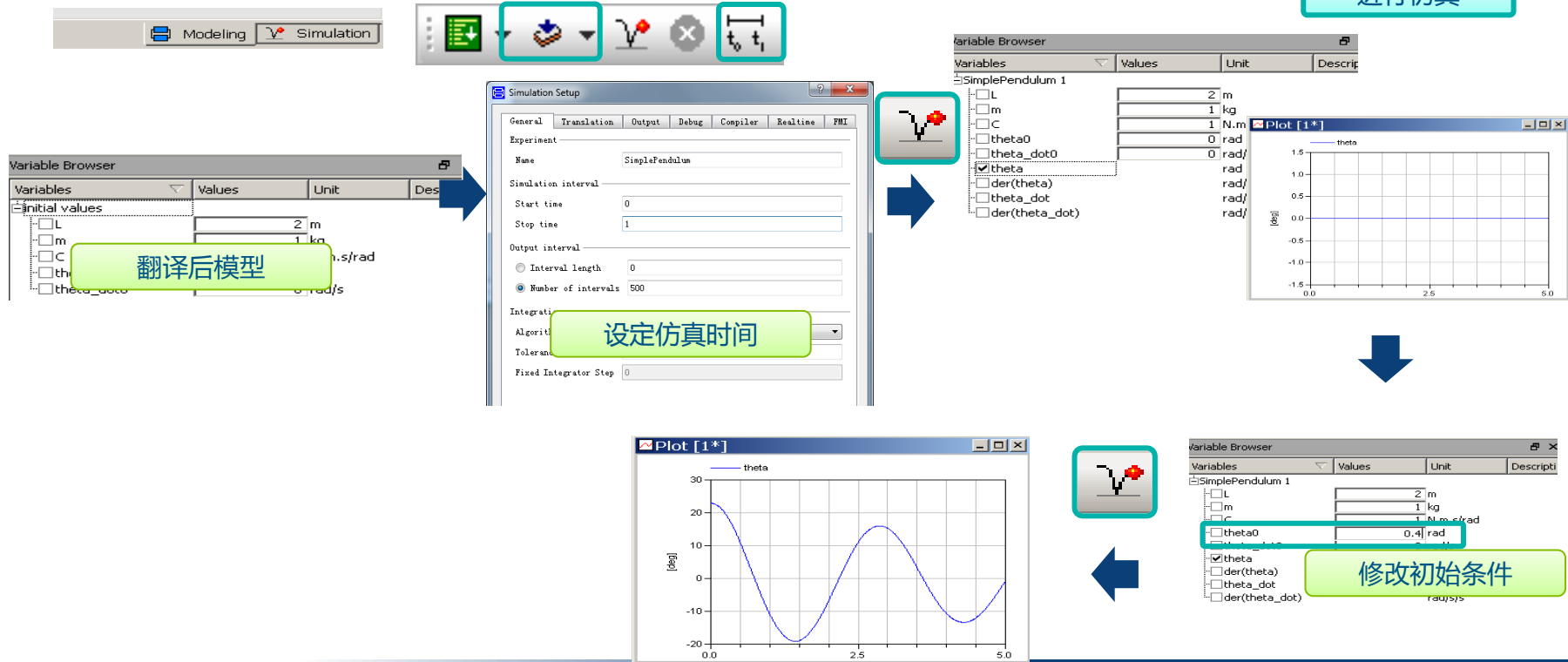
模型建立完毕进行语法及方程/未知数检查 Check



练习二：单摆(8/10)

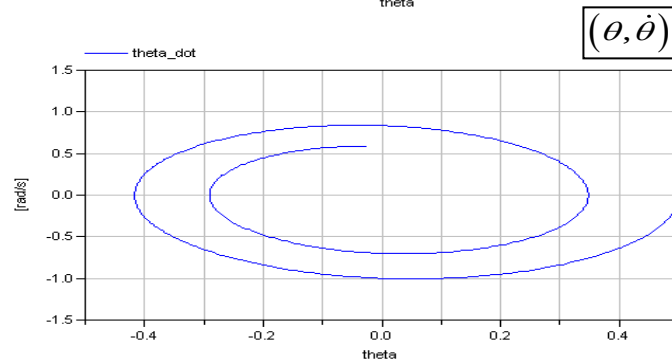
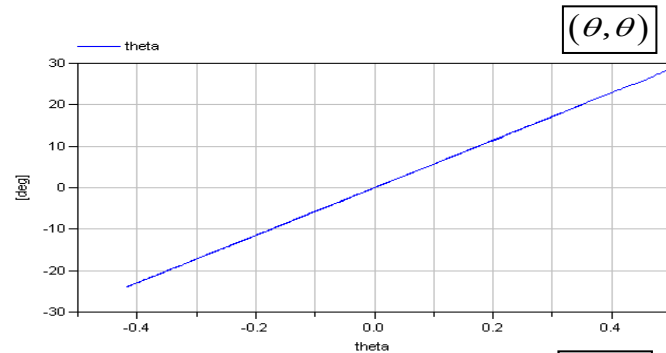
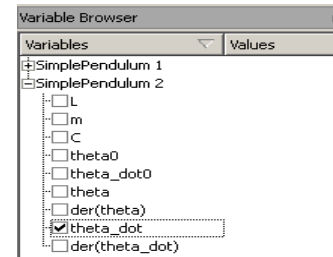
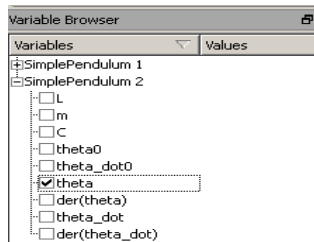
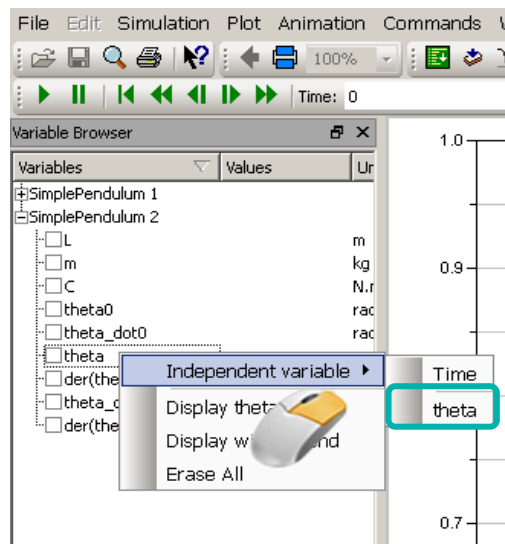
5

进行仿真



练习二：单摆(9/10)

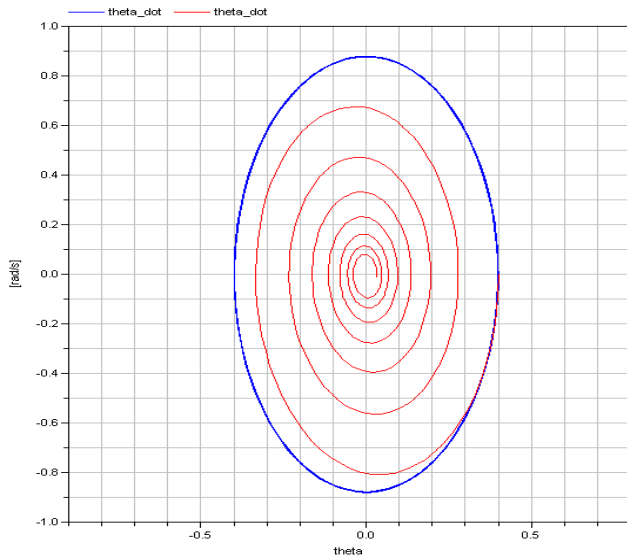
不一定以时间作为坐标轴，可选择任意参数/变量作为横坐标轴以绘制相图



练习二：单摆(10/10)

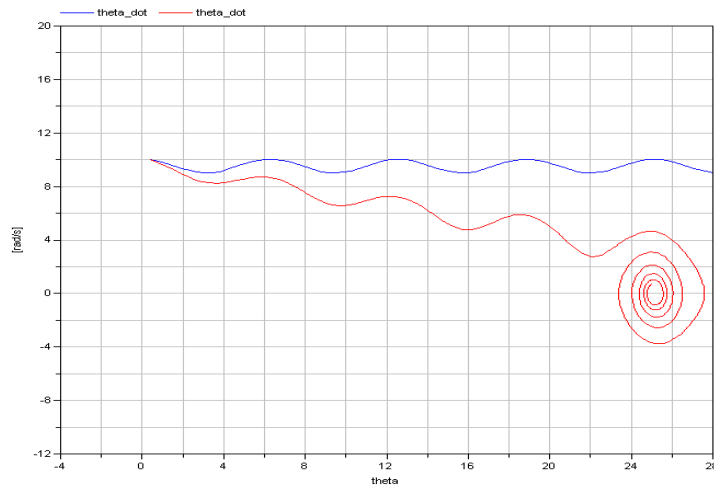
不一定以时间作为坐标轴，可选择任意参数/变量作为横坐标轴以绘制相图

$$(\theta, \dot{\theta})$$



$$\theta_0 = 0.4 \text{ rad}; \dot{\theta}_0 = 0 \text{ rad} \cdot \text{s}^{-1};$$

$$c = 0 \text{ N} \cdot \text{m} \cdot \text{s} \cdot \text{rad}^{-1}; c = 1 \text{ N} \cdot \text{m} \cdot \text{s} \cdot \text{rad}^{-1}$$

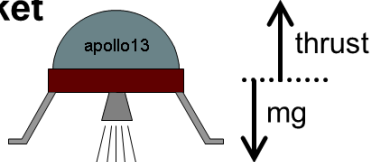


$$\theta_0 = 0.4 \text{ rad}; \dot{\theta}_0 = 10 \text{ rad} \cdot \text{s}^{-1};$$

$$c = 0 \text{ N} \cdot \text{m} \cdot \text{s} \cdot \text{rad}^{-1}; c = 1 \text{ N} \cdot \text{m} \cdot \text{s} \cdot \text{rad}^{-1}$$

练习三：月球登陆器(1/4)

Rocket



$$acceleration = \frac{thrust - mass \cdot gravity}{mass}$$

$$mass' = -massLossRate \cdot abs(thrust)$$

$$altitude' = velocity$$

$$velocity' = acceleration$$

new model

parameters (changeable before the simulation)

floating point type

differentiation with regards to time

```
class Rocket["rocket class"]
  parameter String name;
  Real mass(start=1038.358);
  Real altitude(start= 59404);
  Real velocity(start=-2003);
  Real acceleration;
  Real thrust; // Thrust force on rocket
  Real gravity; // Gravity forcefield
  parameter Real massLossRate=0.000277;
equation
  (thrust-mass*gravity)/mass = acceleration;
  der(mass) = -massLossRate * abs(thrust);
  der(altitude) = velocity;
  der(velocity) = acceleration;
end Rocket;
```

declaration comment

start value

name + default value

mathematical equation (acausal)

Courtesy of Peter Fritszon "Modelica – A Cyber-Physical Modeling Language for Systems Engineering"

练习三：月球登陆器(2/4)

A class declaration creates a *type name* in Modelica

```
class CelestialBody
  constant Real g = 6.672e-11;
  parameter Real radius;
  parameter String name;
  parameter Real mass;
end CelestialBody;
```



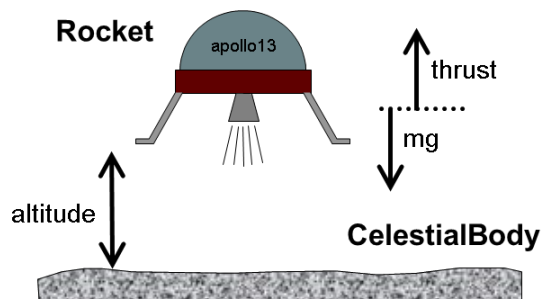
An *instance* of the class can be declared by *prefixing* the type name to a variable name

```
...
CelestialBody moon;
...
```

The declaration states that **moon** is a variable containing an object of type **CelestialBody**

Courtesy of Peter Fritszon "Modelica – A Cyber-Physical Modeling Language for Systems Engineering"

练习三：月球登陆器(3/4)



$$apollo.gravity = \frac{moon.g \cdot moon.mass}{(apollo.altitude + moon.radius)^2}$$

only access
inside the class

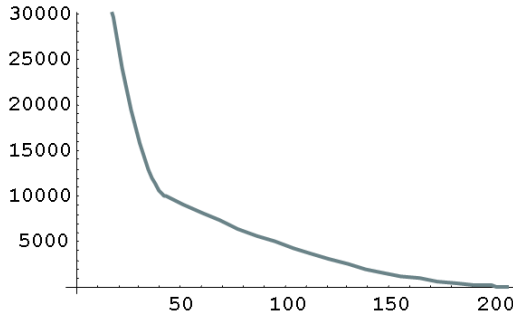
access by dot
notation outside
the class

```
class MoonLanding
  parameter Real force1 = 36350;
  parameter Real force2 = 1308;
  protected
    parameter Real thrustEndTime = 210;
    parameter Real thrustDecreaseTime = 43.2;
  public
    Rocket apollo name="apollo13";
    CelestialBody moon name="moon", mass=7.382e22, radius=1.738e6;
  equation
    apollo.thrust = if (time < thrustDecreaseTime) then force1
                     else if (time < thrustEndTime) then force2
                     else 0;
    apollo.gravity=moon.g*moon.mass/(apollo.altitude+moon.radius)^2;
end MoonLanding;
```

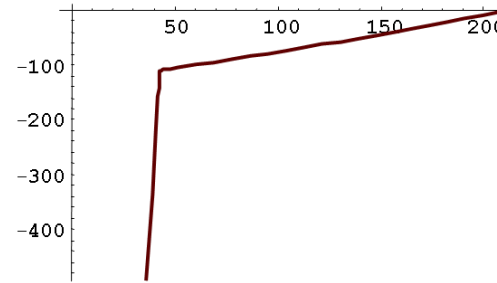
Courtesy of Peter Fritszon "Modelica – A Cyber-Physical Modeling Language for Systems Engineering"

练习三：月球登陆器(4/4)

```
simulate(MoonLanding, stopTime=230)  
plot(apollo.altitude, xrange={0,208})  
plot(apollo.velocity, xrange={0,208})
```



It starts at an altitude of 59404 (not shown in the diagram) at time zero, gradually reducing it until touchdown at the lunar surface when the altitude is zero



The rocket initially has a high negative velocity when approaching the lunar surface. This is reduced to zero at touchdown, giving a smooth landing

Courtesy of Peter Fritszon "Modelica – A Cyber-Physical Modeling Language for Systems Engineering"

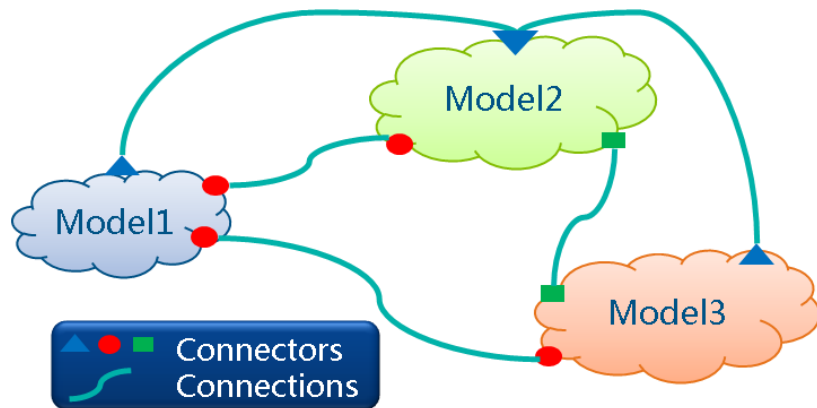
Connectors 接口(1/6)

Models 模型 , Connectors 接口 , Connections 连接关系

- ❖ 连接关系指定了部件间交互的信息。连接的接口包含了所有描述交互的所有特性。例如电气部件之间会传递电压和电流，又或者动力总成部件间传递角度和扭矩。
- ❖ 对象间的连接关系在一个类的行为方程部分用连接方程(“connect-equation”)进行表示，如下：
`connect ( " component_reference "; " component_reference " );`
- ❖ 连接方程语句实际上会根据接口类型和传递的变量翻译成一系列方程
 - ❖ 对于Non-flow 变量，翻译时生成等式方程
 - ❖ 对于Flow 变量，翻译时生成零和方程

Non-flow和Flow变量 (流体接口还有stream变量)

```
connector Pin "Pin of an electrical component"  
  Modelica.SIunits.Voltage v "Potential at the pin" a;  
  flow Modelica.SIunits.Current i "Current flowing into the pin" a;  
a  
end Pin;
```



Connectors 接口(2/6)

一维平动机械连接范例

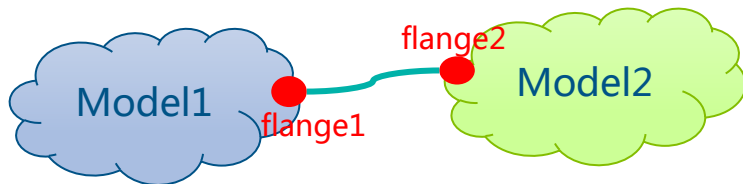
```
connector Flange_a  
  "{left} 1D translational flange (flange axis directed INTO cut plane, e. g. from left to right)"
```



Non-Flow Variable

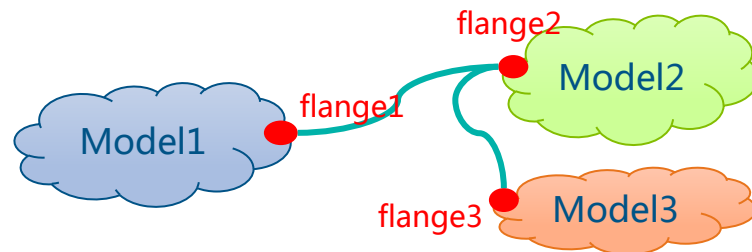
```
SI.Position s "absolute position of flange";  
flow SI.Force f "cut force directed into flange";  
end Flange_a;
```

Flow Variable



```
connect ( flange1, flange2 );
```

$flange1.s = flange2.s$ 位移相等
 $flange1.f + flange2.f = 0$ 力总和为零



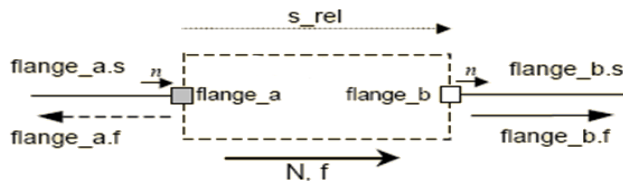
```
connect ( flange1, flange2 );  
connect ( flange2, flange3 );
```

$flange1.s = flange2.s$
 $flange2.s = flange3.s$ 位移相等
 $flange1.f + flange2.f + flange3.f = 0$ 力总和为零

Connectors 接口(3/6)

基于接口进行延展(extends)机制构造的基类(base class)范例

Base class: partial compliant



```
partial model PartialCompliant
    "Compliant connection of two translational 1D flanges"

    Flange_a flange_a "Left flange of compliant 1-dim. translational component" a;
    Flange_b flange_b "Right flange of compliant 1-dim. translational component" b;
    SI.Distance s_rel(start=0) "relative distance (= flange_b.s - flange_a.s)";
    SI.Force f "force between flanges (positive in direction of flange axis R)";
    a;

equation
    s_rel = flange_b.s - flange_a.s;
    flange_b.f = f;
    flange_a.f = -f;
end PartialCompliant;
```



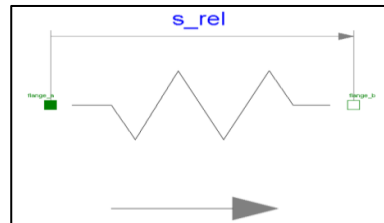
Extended class: spring

```
model Spring "Linear 1D translational spring"
    extends Translational.Interfaces.PartialCompliant;
    parameter SI.TranslationalSpringConstant c(final min=0, start = 1)
        "spring constant ";
    parameter SI.Distance s_rel0=0 "unstretched spring length";

    a;

equation
    f = c*(s_rel - s_rel0);
end Spring;
```

弹簧 = 平移基类 + 胡克定律



Connectors 接口(4/6)

常见基于能量守恒预定义接口

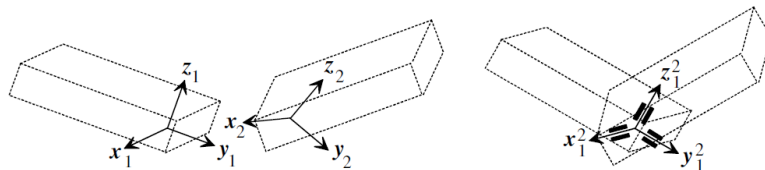
Domain Type	Potential	Flow	Carrier	Modelica Library
Electrical	Voltage	Current	Charge	Electrical. Analog
Translational	Position	Force	Linear momentum	Mechanical. Translational
Rotational	Angle	Torque	Angular momentum	Mechanical. Rotational
Magnetic	Magnetic potential	Magnetic flux rate	Magnetic flux	
Hydraulic	Pressure	Volume flow	Volume	HyLibLight
Heat	Temperature	Heat flow	Heat	HeatFlow1D
Chemical	Chemical potential	Particle flow	Particles	
Pneumatic	Pressure	Mass flow	Air	PneuLibLight

Connectors 接口(5/6)

多体动力学部件接口

3D mechanical connectors

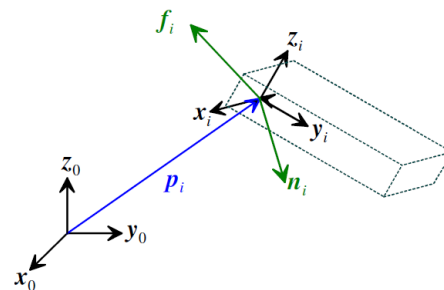
- The connection of two mechanical connectors is equivalent to a **welding**.



Effort $\Rightarrow$ position₁ = position₂

Flow $\Rightarrow$ force₁ + force₂ = 0

Connector variables



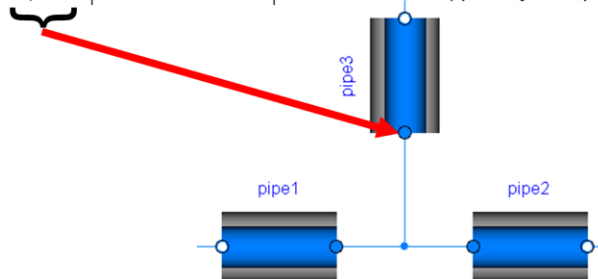
- Effort variables:
 - Position p_i [3×1]
 - “Orientation”** [?]
- Flow variables:
 - Force f_i [3×1]
 - Torque n_i [3×1]

Connectors 接口(6/6)

流体部件接口

Examples of an interface for fluid components

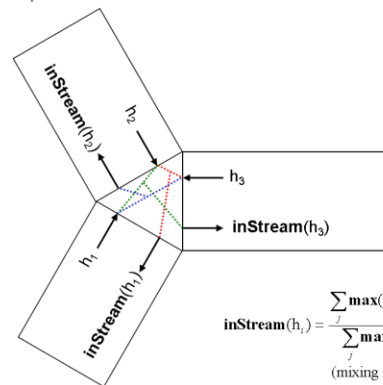
p	absolute pressure	potential	variable
m_flow	mass flow rate	flow	variable (describes transport of matter)
h	specific enthalpy	stream	variable (quantity transported with matter)
X _i	mass fraction	stream	variable (quantity transported with matter)



A **stream** variable h_i of a connector i is associated with
 (a) a **flow** variable m_flow (" $0 = \sum m_flow_i$ ") and
 (b) a **stream balance** equation (" $0 = \sum m_flow_i * \text{<upstream value of } h_i \text{>}$ ")

Reliable handling of bi-directional flow:

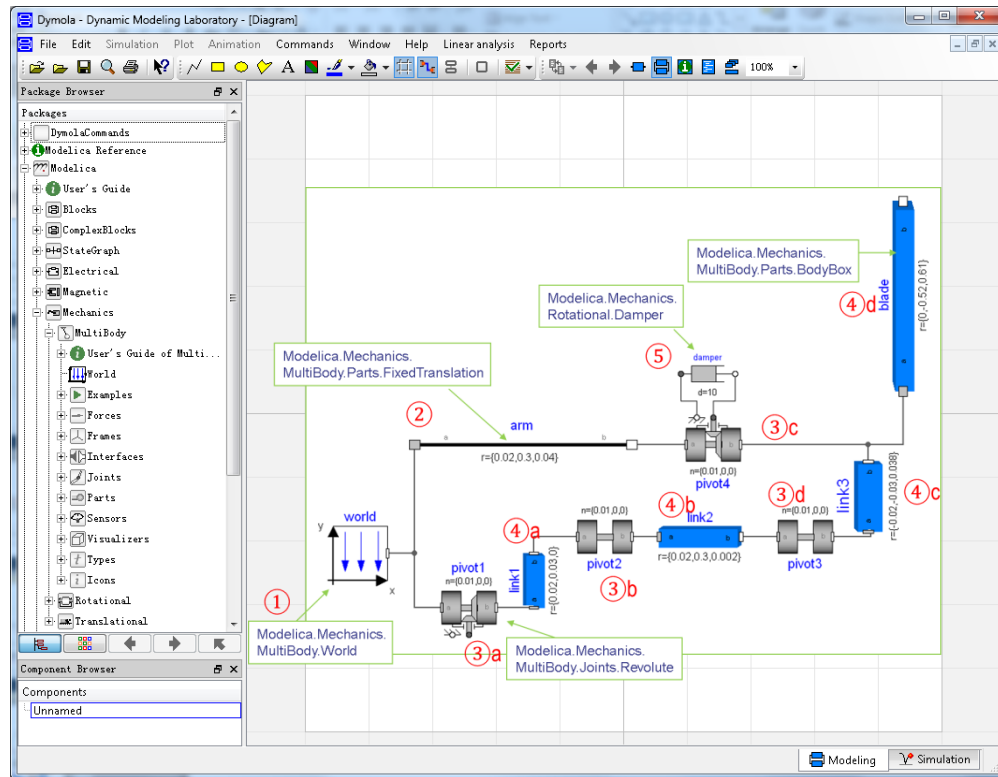
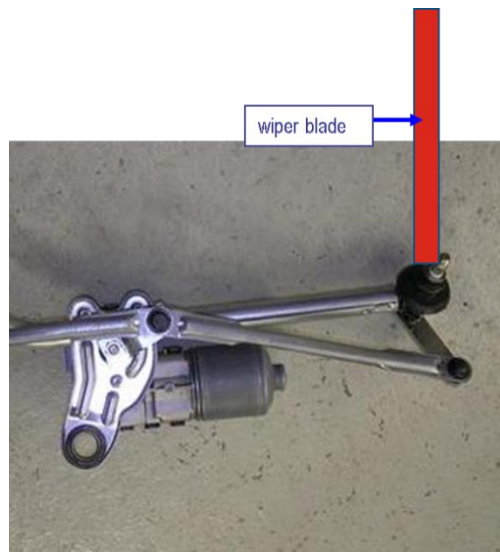
	<i>value of stream variable h_i in connector i</i>
h_i	if flow from component to connection point ($m_flow_i \leq 0$)
inStream (h_i)	if flow from connection point to component ($m_flow_i > 0$)
actualStream (h_i)	for both flow directions (= if $m_flow_i > 0$ then inStream (h_i) else h_i) only to be used when absolutely necessary(!)



$$\text{inStream}(h_i) = \frac{\sum_j \max(-m_flow_j, \epsilon) \cdot h_j}{\sum_j \max(-m_flow_j, \epsilon)} \quad (j \neq i, m_flow.\min_j < 0)$$

(mixing value for $m_flow_i > 0$)

练习四：雨刮器(1/5)



练习四：雨刮器(2/5)

World ①

General Tab:

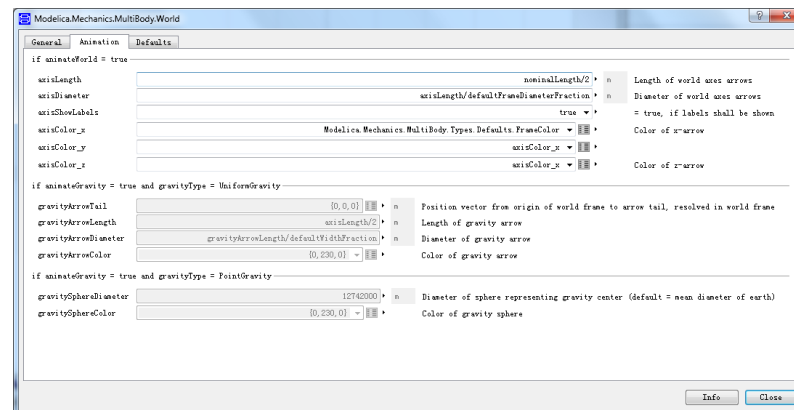
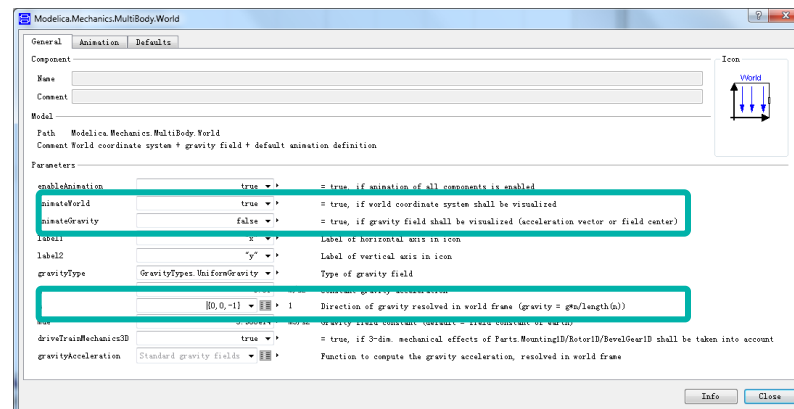
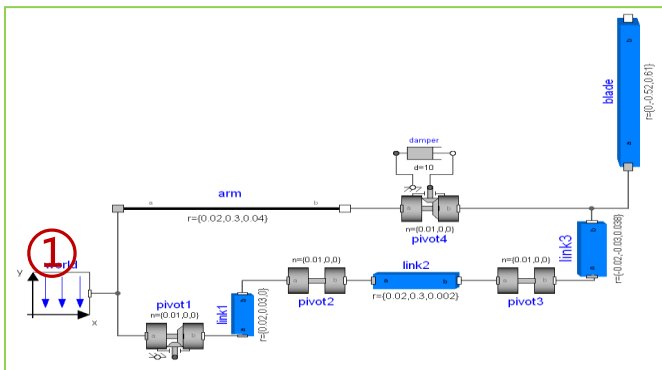
animateWorld: **true**

animateGravity: **false**

$n=\{0,0,-1\}$

Animation Tab:

axisLength=**nominalLength/4**



Arm ② Modelica.Mechanics.MultiBody.Parts.FixedTranslation

$$r = D - A = \{0.02, 0.3, 0.04\}$$

Links: [Modelica.Mechanics.MultiBody.Parts.BodyBox](#)

link1 ④a :

$$r = B - A = \{0.02, 0.03, 0\}$$

width= 0.01

link2 ④b :

$$r = C - B = \{0.02, 0.3, 0.002\}$$

width= 0.01

link3 ④c :

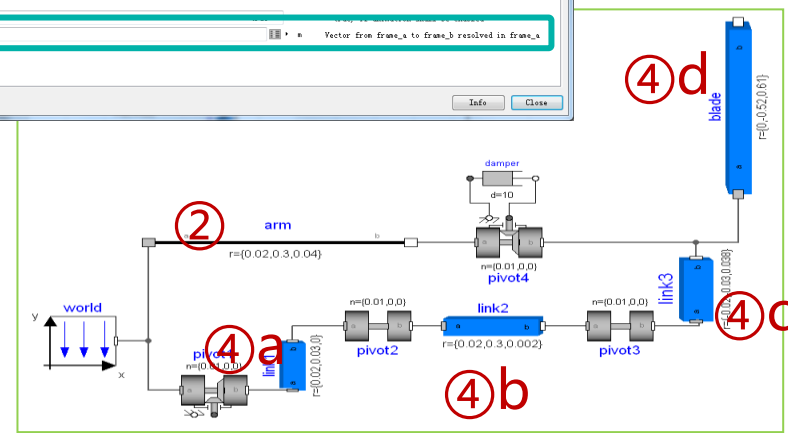
$$r = D - C = \{-0.02, -0.03, 0.038\}$$

width= 0.01

Blade ④d : [Modelica.Mechanics.MultiBody.Parts.BodyBox](#)

$$r = E - D = \{0, -0.52, 0.61\}$$

density=400



练习四：雨刮器(4/5)

Pivot: ② Modelica.Mechanics.MultiBody.Joints.Revolute

General Tab

$n = \{0.01, 0, 0\}$

useAxisFlange: ③a、③c

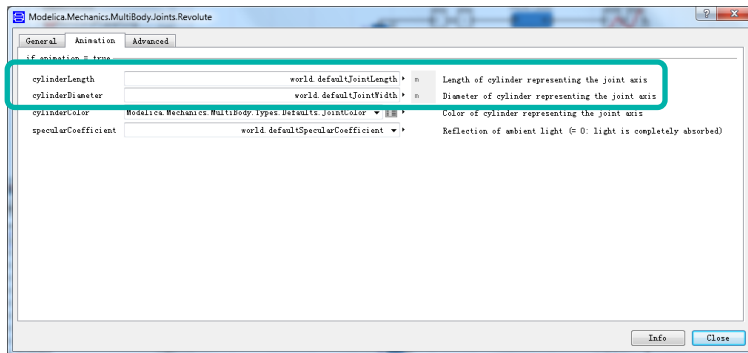
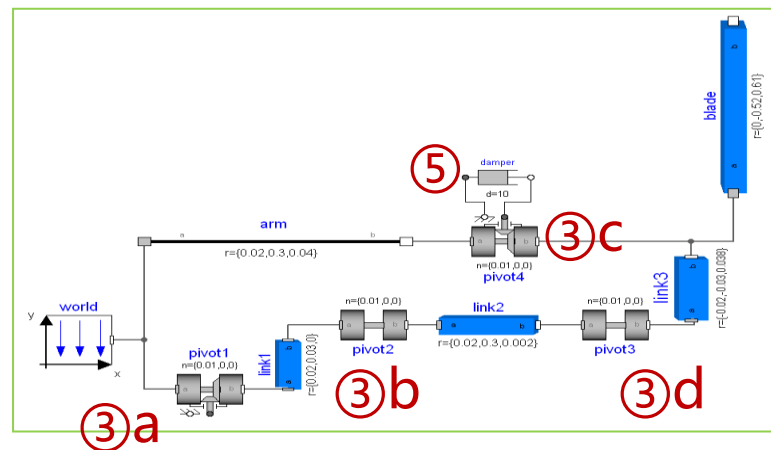
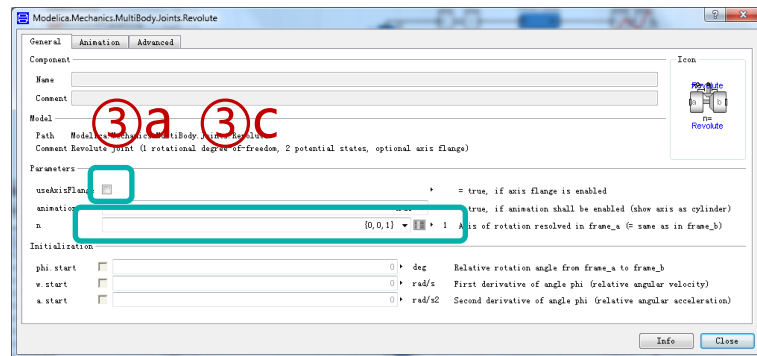
Animation Tab ③a-d

CylinderLength: 0.01

CylinderDiameter: 0.01

Damper ⑤ Modelica.Mechanics.Rotational.Components.Damper

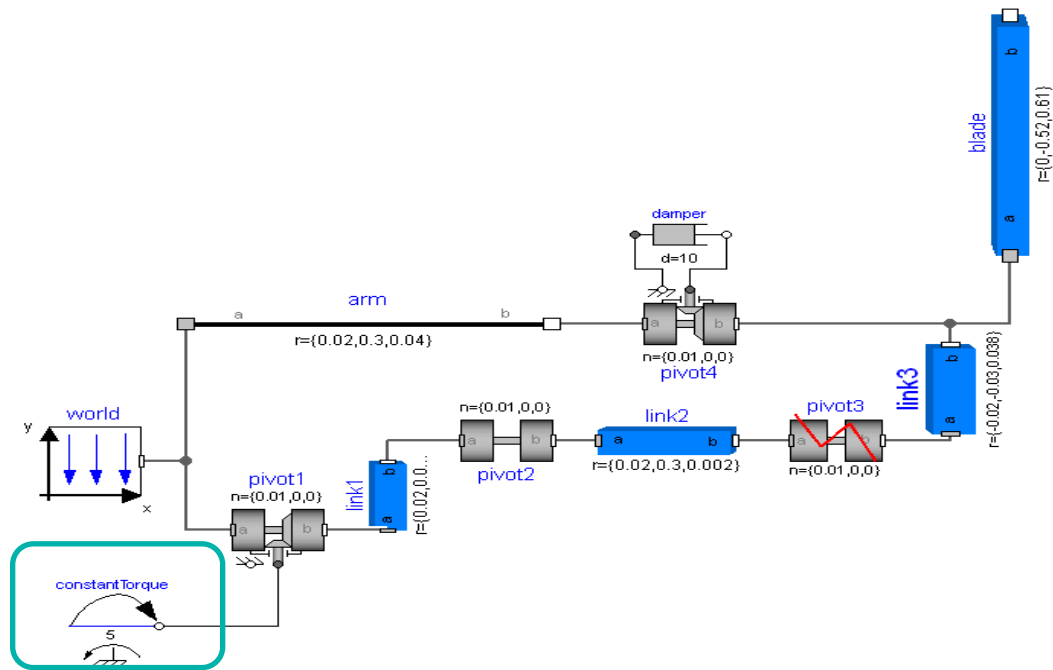
$d=10$



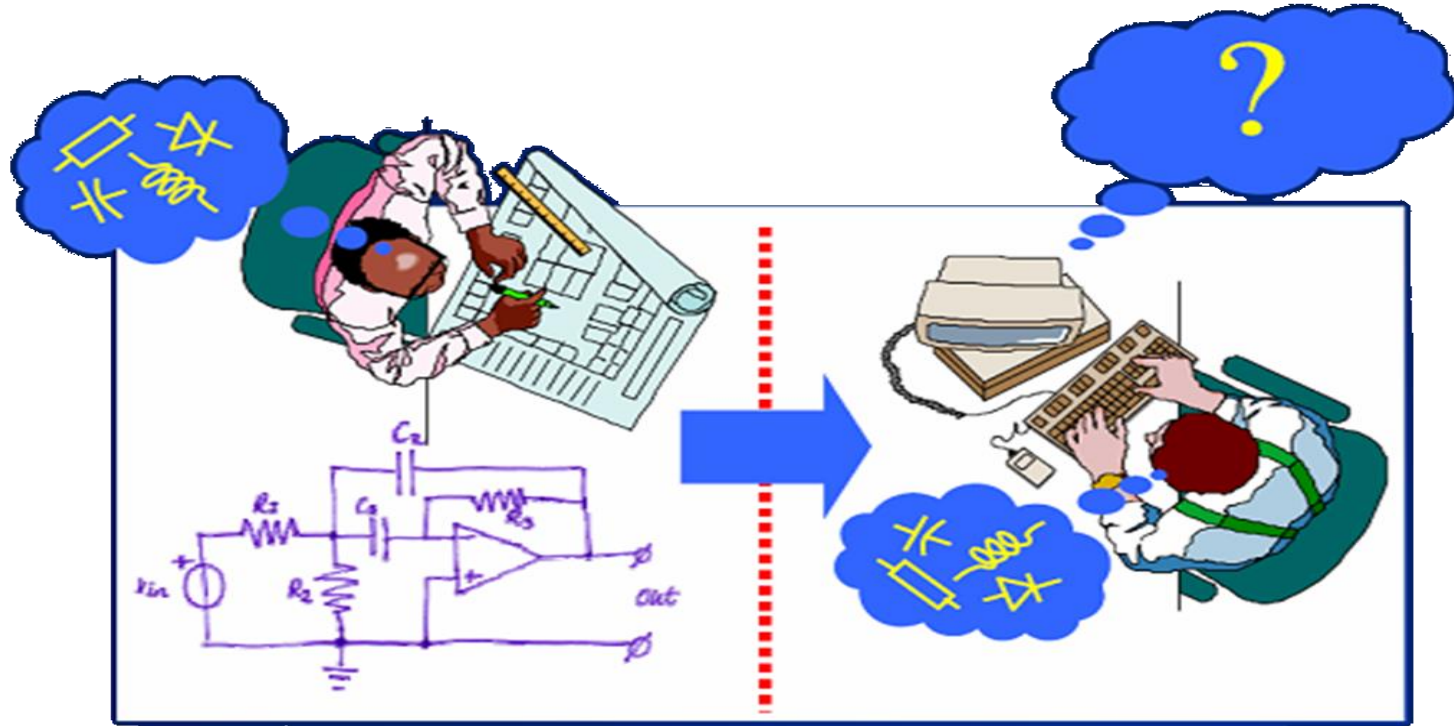
练习四：雨刮器(5/5)

Driving torque : Modelica.Mechanics.Rotational.ConstantTorque

tau_constant = 5 Nm



Acausality 非因果性(1/5)



Acausality 非因果性(2/5)

工程师可专注于本专业利用**方程**进行**物理建模**而非求解过程（因果）建模

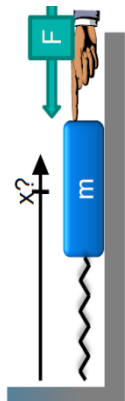


专业工程师



求解工具

弹簧-质点
问题



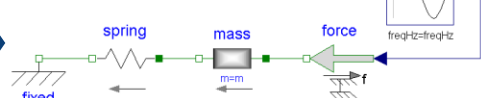
非因果方式建模

1 Specify/reuse **component** behavior using equations

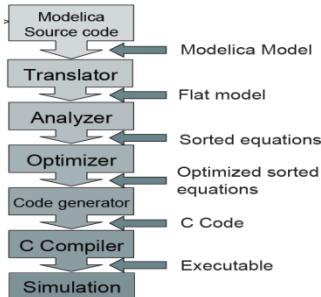
mass

```
equation
v = der(s);
a = der(v);
m*a = flange_a.f + flange_b.f;
```

2 Acausal declarative **system** physical model



3

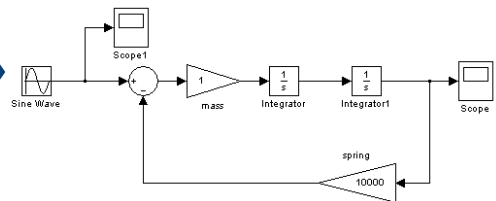


因果方式建模

1 **System** behavior
 $m \cdot \ddot{x} + k \cdot x = F$

2 Transfer function
$$X(s) = \frac{F(s)}{ms^2 + k}$$

3 Causal **algorithm model** 算法模型



4 Solving algorithm code generation & simulation

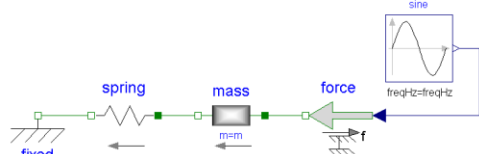
Acausality 非因果性(3/5)

工程师可专注于本专业利用**方程**进行**物理建模**而非求解过程（因果）建模

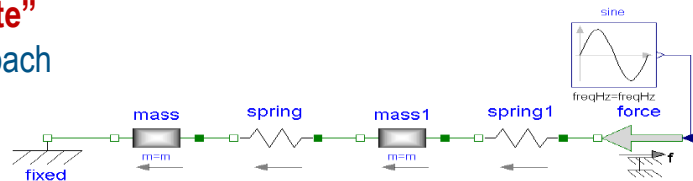
Acausal vs. Causal



非因果方式建模

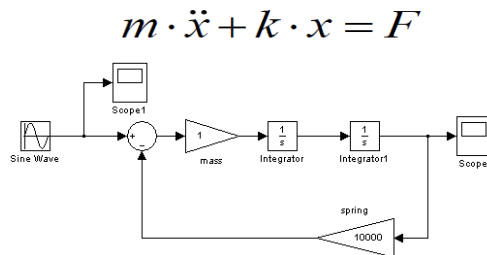


“Copy-paste”
reuse approach



对于物理系统，非因果建模方式大大提高模型重用

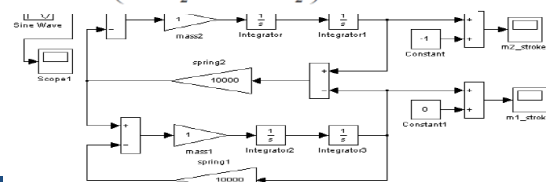
因果方式建模



$$m \cdot \ddot{x} + k \cdot x = F$$

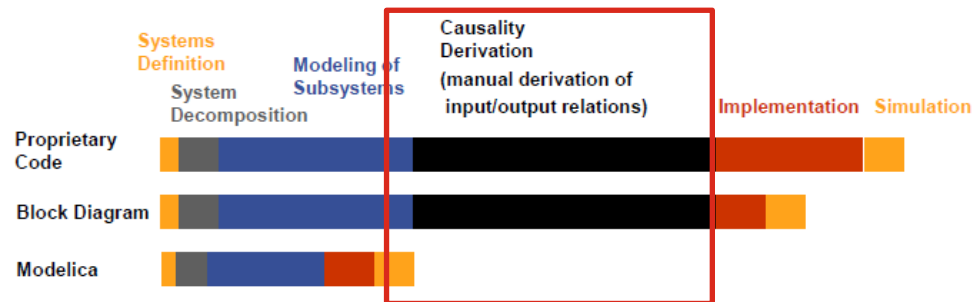
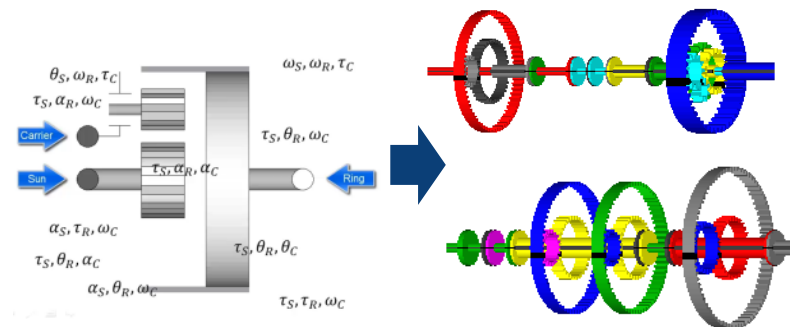
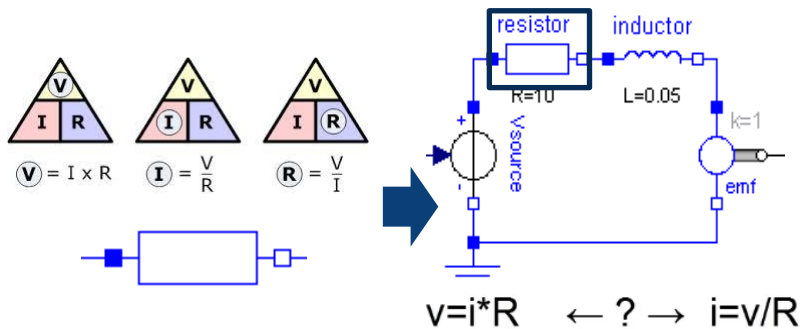
Effort re-analyzing
system dynamics

$$\begin{pmatrix} \ddot{x}_1 \\ \ddot{x}_2 \end{pmatrix} = \begin{pmatrix} -\frac{k_1 + k_2}{m_1} & \frac{k_2}{m_1} \\ \frac{k_2}{m_2} & -\frac{k_2}{m_2} \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \end{pmatrix} + \begin{pmatrix} 0 \\ \frac{F}{m_2} \end{pmatrix}$$



Acausality 非因果性(4/5)

采用因果方式对复杂物理系统将耗费大量因果性推导工作



Acausality 非因果性(5/5)

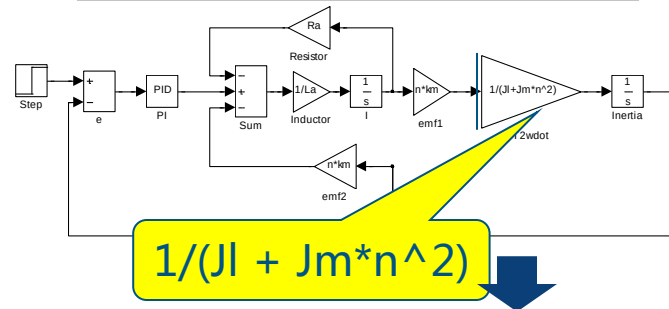
Knowledge/Library



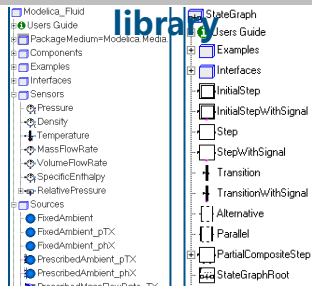
C/Fortran code

[illegible]

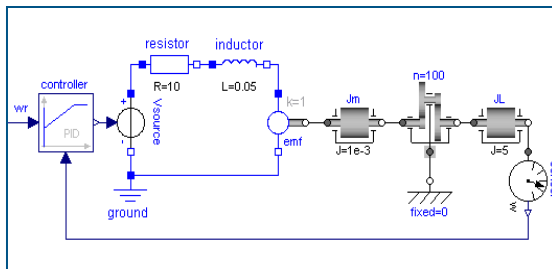
Complex algorithm by expert



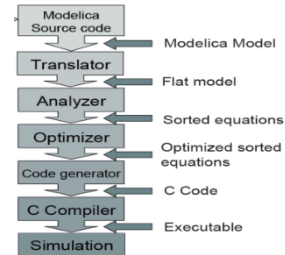
Knowledge acquirement, reuse, cooperation = "Modelized"



Easy Modeling

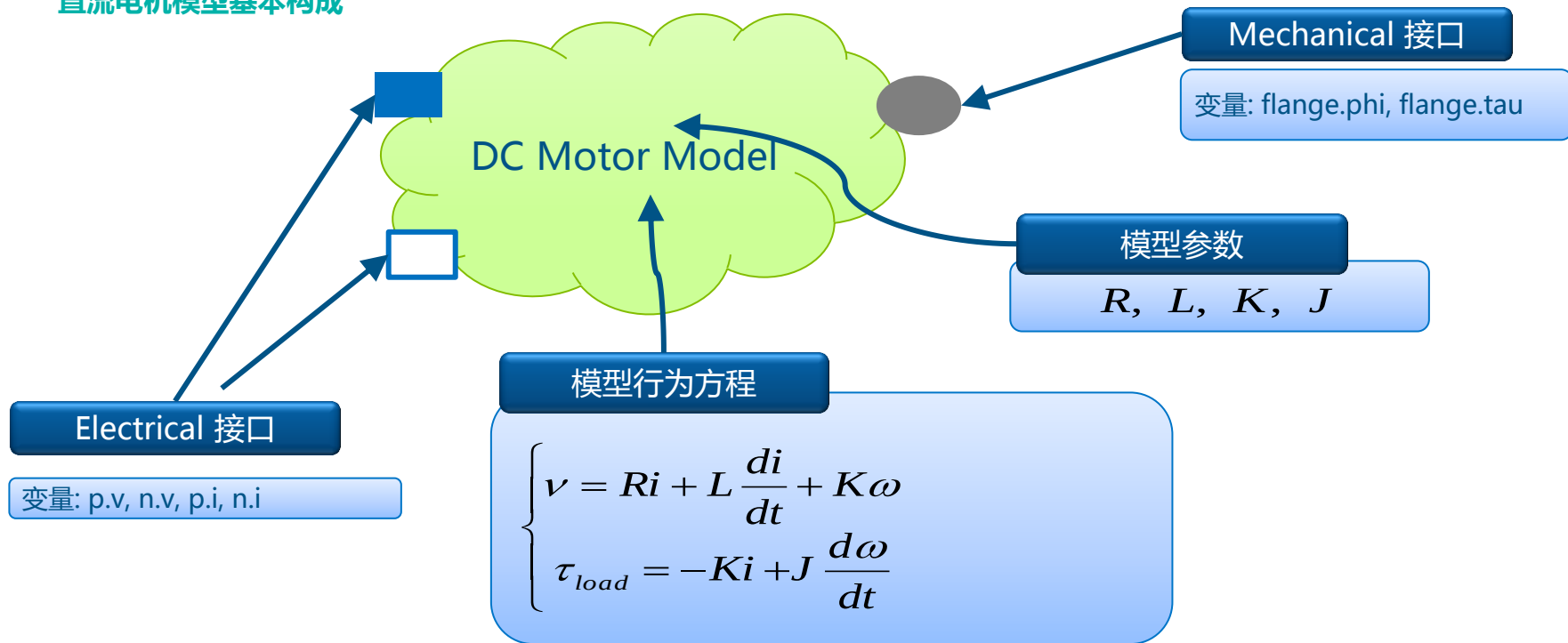


Model processing/ symbolic manipulation



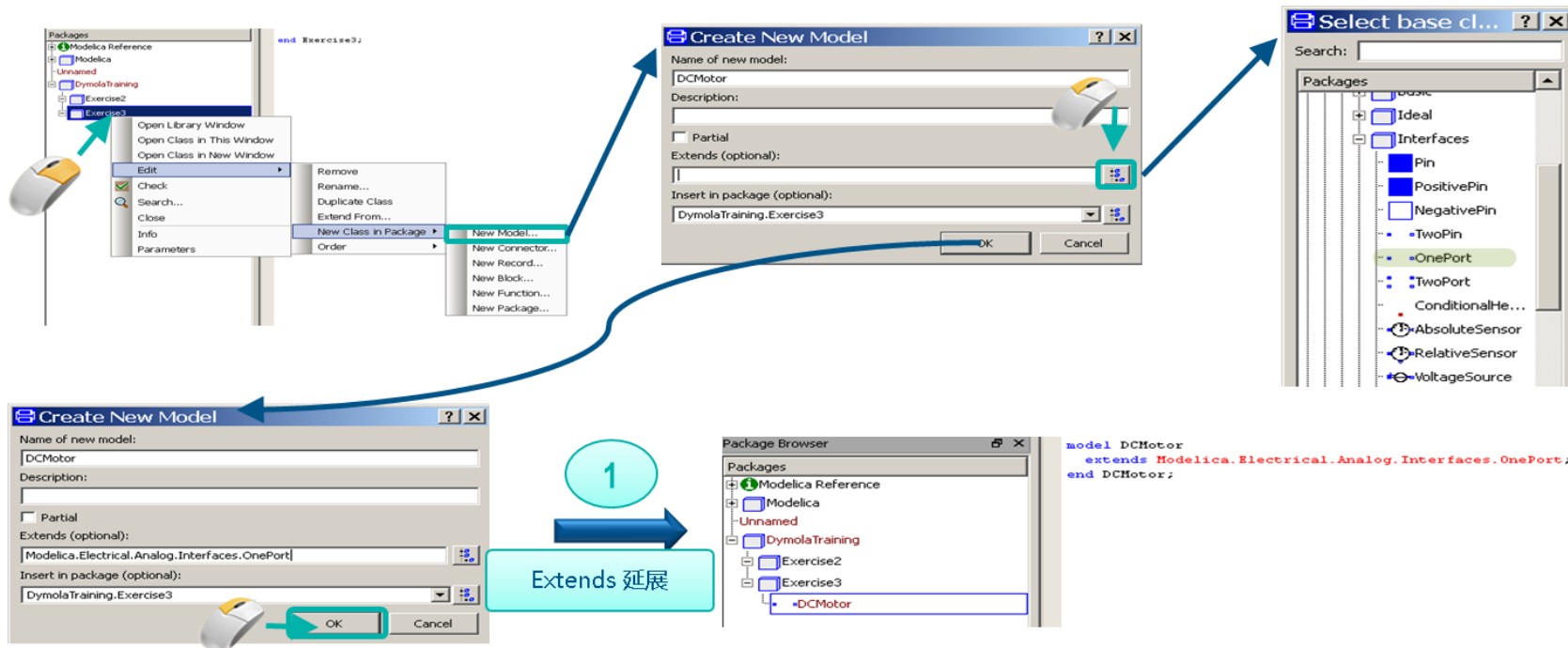
练习五：直流电机(1/9)

直流电机模型基本构成



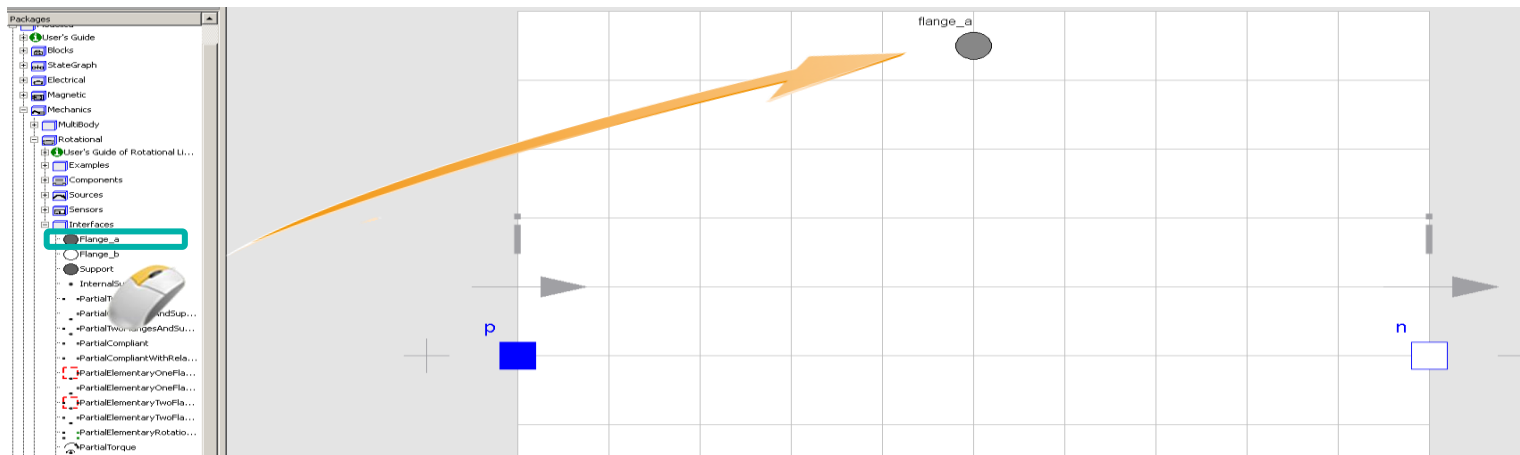
练习五：直流电机(2/9)

创建Exercise5 package并通过延展 Modelica.Electrical.Analog.Interfaces.OnePort 得到电机模型



练习五：直流电机(3/9)

浏览Modelica标准库MSL中找到Modelica.Mechanics.Rotational.Interfaces.Flange_a 拖拽到电机模型



注意“a;”展开后是annotation代码，指定了部件在Dymola中图形符号。如果不采用拖拽而是直接输入部件代码则部件不会在Dymola中有图形显示



```
model DCMotor
  extends Modelica.Electrical.Analog.Interfaces.OnePort;
  Modelica.Mechanics.Rotational.Interfaces.Flange_a flange_a
  a;
end DCMotor;
```

练习五：直流电机(4/9)

在Modelica代码界面将参数/变量及行为方程补充完整

```
model DCMotor
  extends Modelica.Electrical.Analog.Interfaces.OnePort;

  /*****
   * PARAMETERS
   *****/
  parameter Modelica.SIunits.Inductance L=0.01 "Inductance of the DC Motor";
  parameter Modelica.SIunits.Resistance R=0.05 "Resistance of the DC Motor";
  parameter Modelica.SIunits.ElectricalTorqueConstant K=0.3
    "Torque constant of the DC Motor";
  parameter Modelica.SIunits.Inertia J=0.2 "Rotor inertia of the DC Motor";

  /*****
   * INTERMEDIATE VARIABLES
   *****/

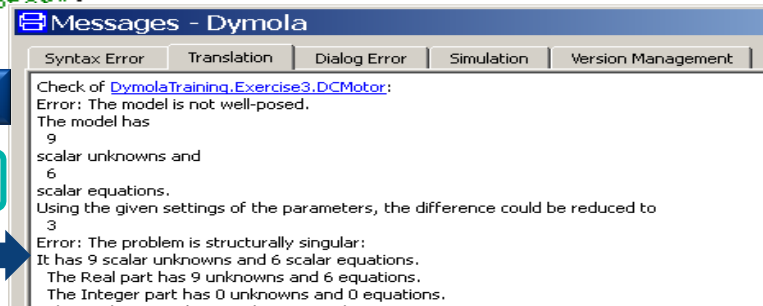
  Modelica.SIunits.AngularVelocity w "Angular Velocity of the DC Motor";

equation
  der(flange_a.phi)=w;

  v=R*i+L*der(i)+K*w;
  flange_a.tau=-K*i+J*der(w);
end DCMotor;
```

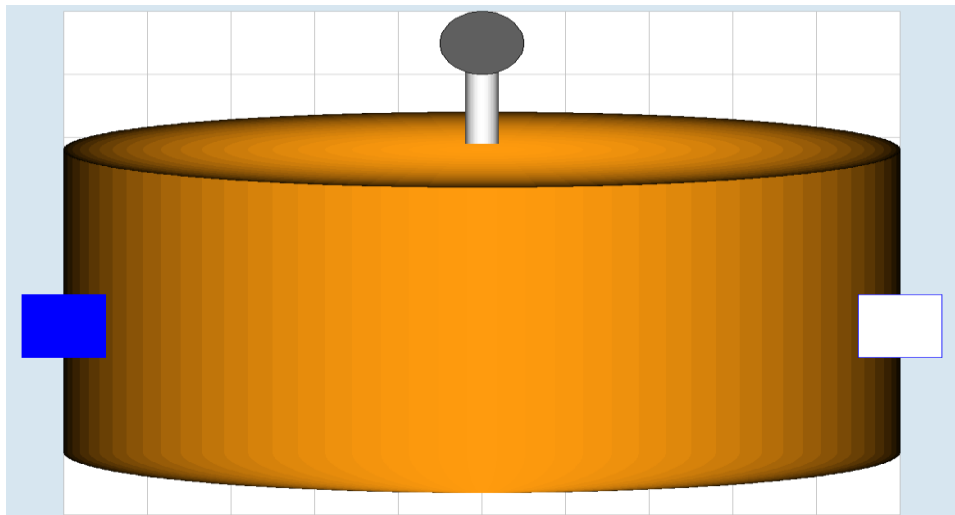
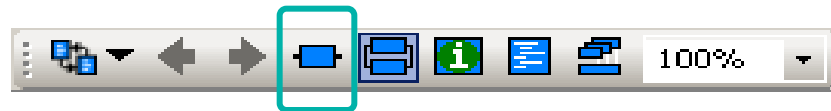
$$\begin{cases} v = Ri + L \frac{di}{dt} + K\omega \\ \tau_{load} = -Ki + J \frac{d\omega}{dt} \end{cases}$$

Check



练习五：直流电机(5/9)

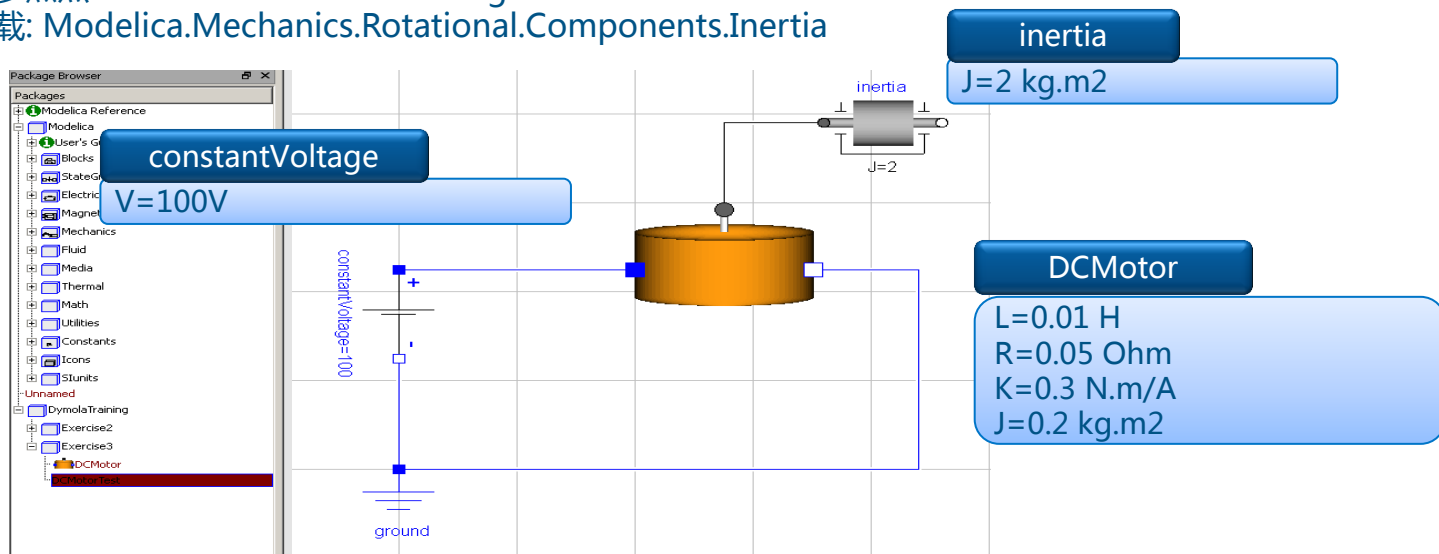
切换至模型的图标界面绘制模型显示图标



练习五：直流电机(6/9)

在当前package下新建DCMotorTest模型，拖拽DCMotor及Modelica标准库中元件至到DCMotorTest

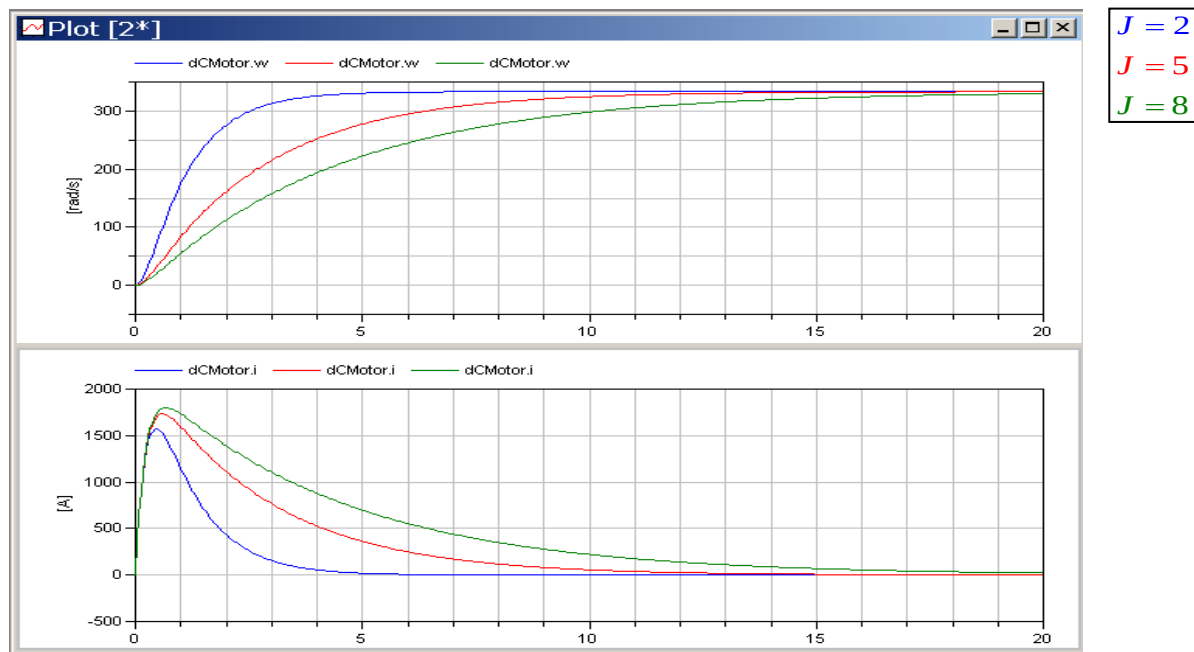
- ❖ 电源: Modelica.Electrical.Analog.Sources.ConstantVoltage
- ❖ 零电势参照点: Modelica.Electrical.Analog.Basic.Ground
- ❖ 惯性负载: Modelica.Mechanics.Rotational.Components.Inertia



拖拽入DCMotorTest的DCMotor是DCMotor的实例(Instance)，实例除参数数值外都不能进行修改。而修改DCMotor类会导致所有DCMotor的实例都跟随改变

练习五：直流电机(7/9)

切换到仿真界面下设置仿真时间为20s并观察在不同情况 (J, L, R, V)下DCMotor的行为

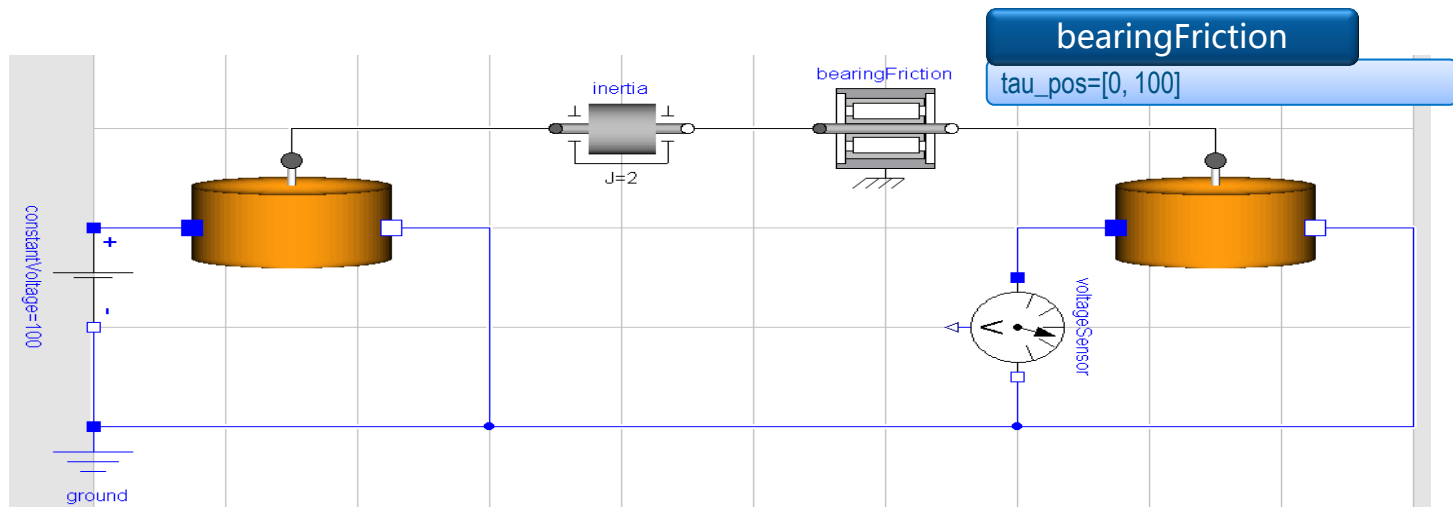


$J = 2$
 $J = 5$
 $J = 8$

练习五：直流电机(8/9)

在当前package下复制(Duplicate)DCMotorTest并更名为TestAcausality，加入新元件并连接如下

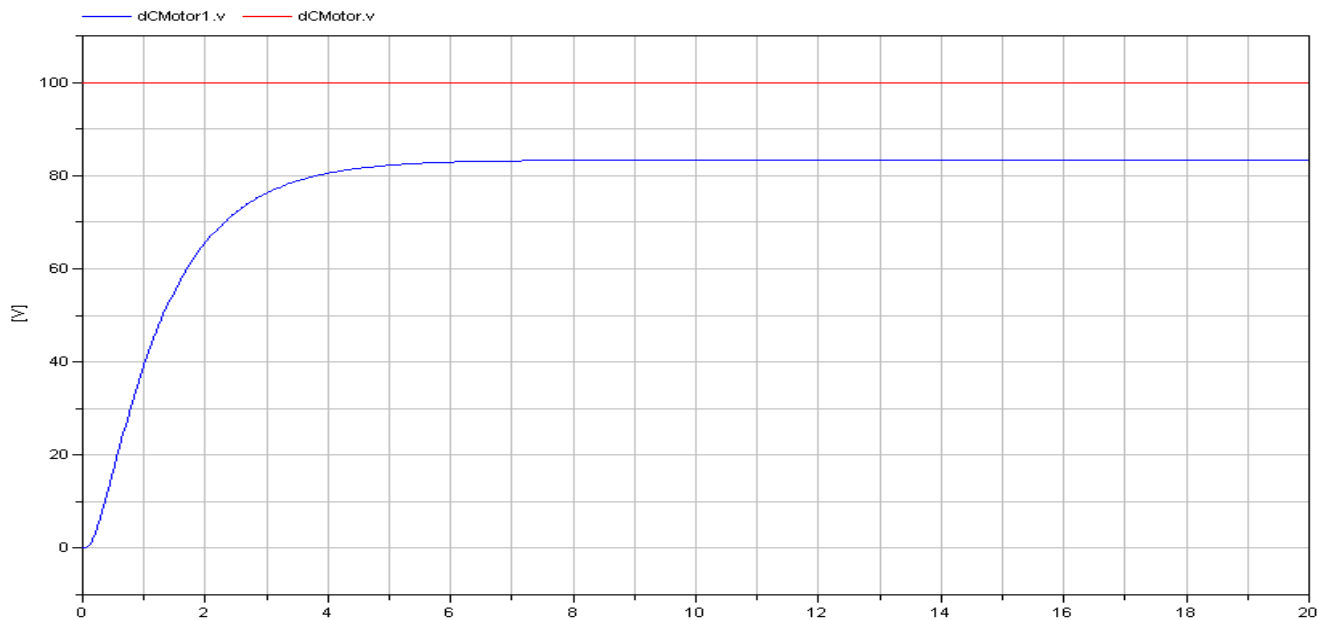
- ❖ 轴承摩擦: Modelica.Mechanics.Rotational.Components.BearingFriction
- ❖ 电压表: Modelica.Electrical.Analog.Sensors.VoltageSensor



新加入的DCMotor由于连接关系改变，在这个试验中变为了发电机

练习五：直流电机(9/9)

切换到仿真界面下设置仿真时间为20s并观察分别作为驱动电机和发电机的DCMotor的电压行为



Discrete Events 离散事件(1/3)

什么是Modelica events事件

- ❖ 时间轴上的瞬间点，事件无持续时间
- ❖ 事件由事件条件由假(false)切换到真(true)时触发



如何利用事件

与事件关联的行为以条件方程表达，在事件发生时被激活或解除。瞬间方程是一种特别的条件方程，只在事件发生时激活。

两类事件

- ❖ 时间事件(Time events)，只依赖于Modelica内嵌的时间常量的数值改变(可预测)，例如：
在time=4s时，打开阀1
- ❖ 状态事件(State events)，依赖于模型状态变量的数值改变(不可预测)，例如：
当用户按下紧急按钮时，关闭阀1

Discrete Events 离散事件(2/3)

与事件相关的Modelica内嵌函数(functions)和操作符(operators)

Title	功能
<code>initial()</code>	返回值在初始化过程为真，其它时间为假。因此在仿真实验开始时触发一个时间事件
<code>terminal()</code>	返回值在完成成果的仿真分析时为真，其它时间为假。因此在仿真实验成功完成时触发一个时间事件

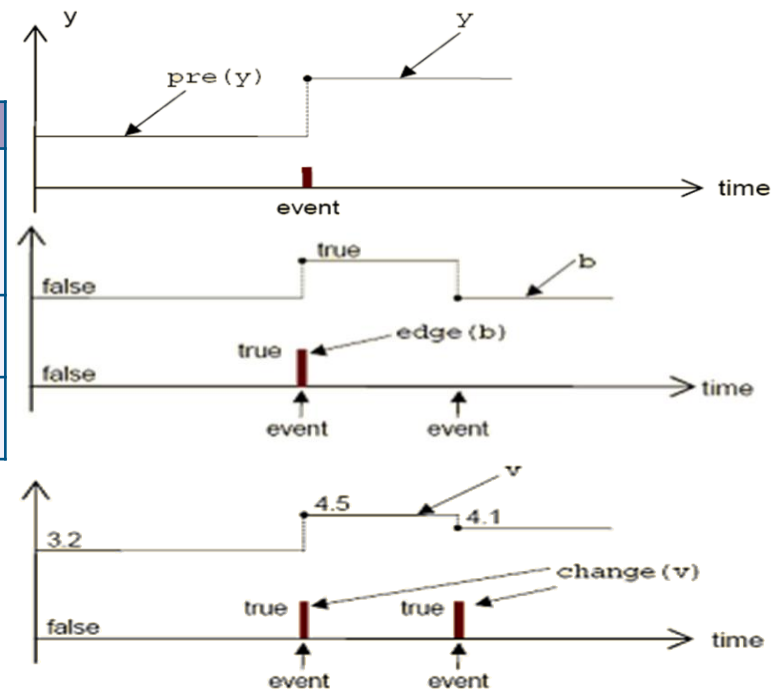


Initial()一般用于离散时间变量的初始化
Terminal()可以用作调用外部函数储存数据

Discrete Events 离散事件(3/3)

与事件相关的Modelica内嵌函数(functions)和操作符(operators)

Title	功能
<code>pre()</code>	Returns the “left limit” $y(t^{pre})$ of variable $y(t)$ at a time instant t . At an event instant, $y(t^{pre})$ is the value of y after the last event iteration at time instant t .
<code>edge()</code>	Is expanded into “(b and not pre(b))” for Boolean variable b .
<code>change()</code>	Is expanded into “(v<>pre(v))” for discrete Real v .



Hybrid Modeling 混合建模(1/5)

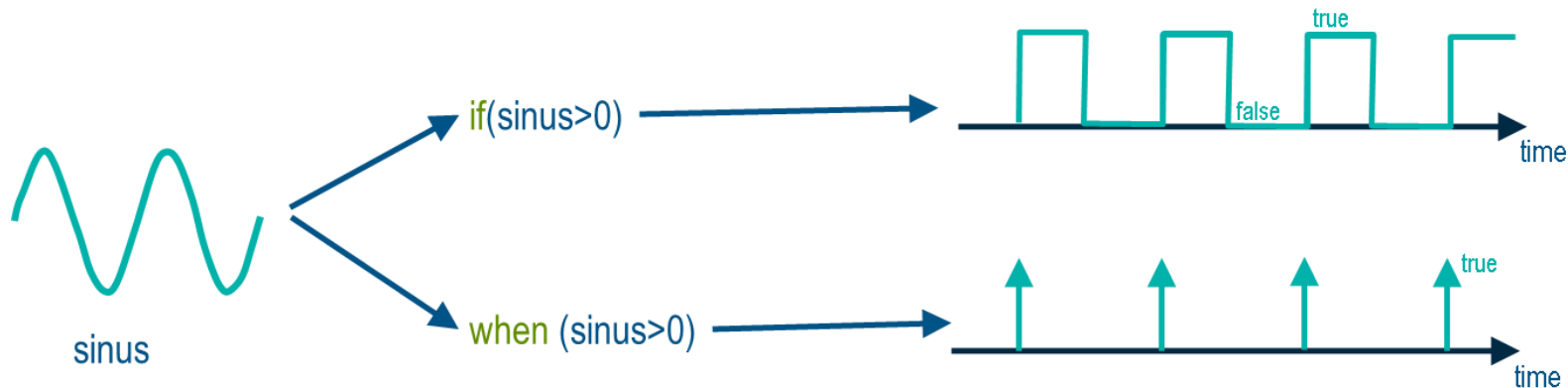
现实物理模型常常包含不连续、离散事件和结构的更改等行为

❖ 例如: 继电器, 开关, 摩擦, 冲击, 数据采集系统等等.

Modelica引入了特别的语言构造使得仿真引擎可以有效的处理这些事件, 如以下两类条件方程

❖ If语句用在包含不同行为的条件模型里。条件方程可能同时包含连续和离散表达式

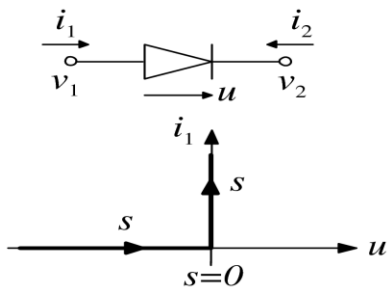
❖ When语句用作表达只有在事件发生时才成立的瞬时方程, 因此其条件方程只包含离散表达式



Hybrid Modeling 混合建模(2/5)

If语句基本构造

```
if condition then  
  
elseif condition then  
  
else  
  
end if;
```



```
model Diode  
...  
equation  
  off = s < 0;  
  If off then  
    u=s;  
  else  
    s=0;  
  end if;  
  i1 = if off then 0 else s;  
end Diode;
```

Hybrid Modeling 混合建模(3/5)

When语句基本构造

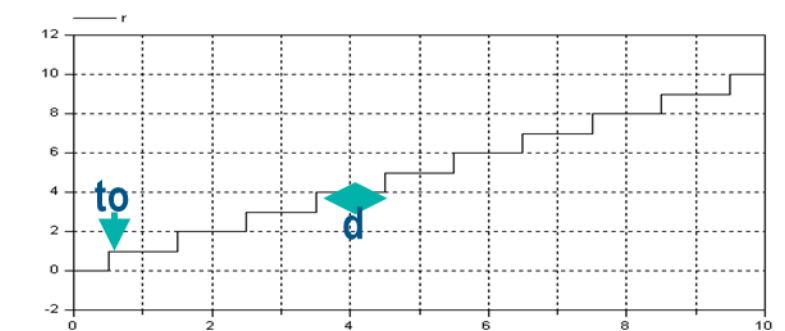
```
when condition then

elsewhen condition then

end when;

end Unnamed;
```

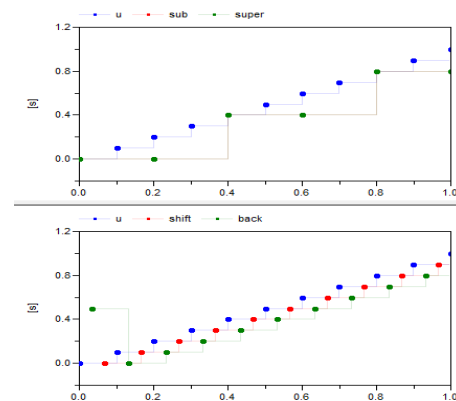
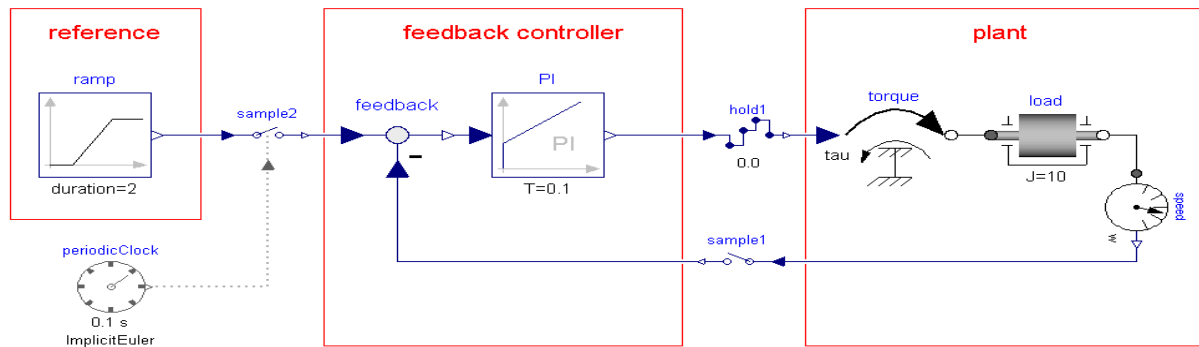
```
class SamplingClock
  parameter Modelica.SIunits.Time to=0.5;
  parameter Modelica.SIunits.Time d=1;
  Boolean clock;
  Integer r;
equation
  clock = sample(to, d);
  when clock then
    r = 1 + pre(r);
  end when;
end SamplingClock;
```



Hybrid Modeling 混合建模(4/5)

Modelica_Synchronous库

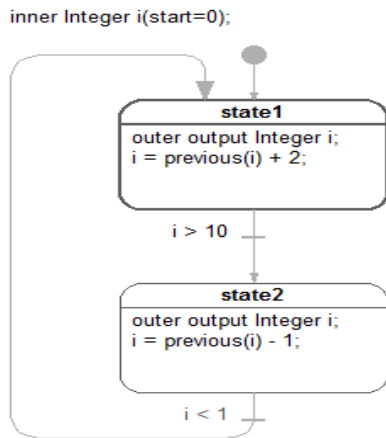
- ❖ 受控对象和控制器分离
- ❖ 连续时间方程和离散时间方程的边界由操作符定义
- ❖ `sample()`: 对连续时间变量进行采样然后返回预定义时钟的离散时间量
- ❖ `hold()`: 通过保持时钟步长内的值将离散时间量转化为连续时间量
- ❖ `sample`操作符根据时钟Clock的定义来决定何时取样



Hybrid Modeling 混合建模(5/5)

Modelica_Synchronous库

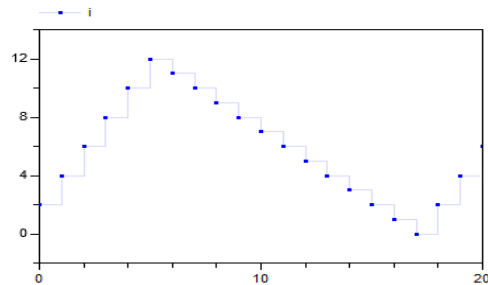
- ❖ 所有不包含连续时间方程或算法的block可以作为状态机的状态
- ❖ 在每个状态机一个和仅有一个实例instance可以用initialState方程定义为初始状态
- ❖ Outer output声明代表了方程可以进入已声明inner的对应变数
- ❖ 状态过渡transition(from, to, condition, immediate, reset, synchronize, priority)



```
model StateMachine1
  inner Integer i(start=0);
  block State1
    outer output Integer i;
  equation
    i = previous(i) + 2;
  end State1;
  State1 statel;

  block State2
    outer output Integer i;
  equation
    i = previous(i) - 1;
  end State2;
  State2 state2;

equation
  initialState(statel);
  transition(statel, state2, i > 10, immediate=false);
  transition(state2, statel, i < 1, immediate=false);
end StateMachine1;
```



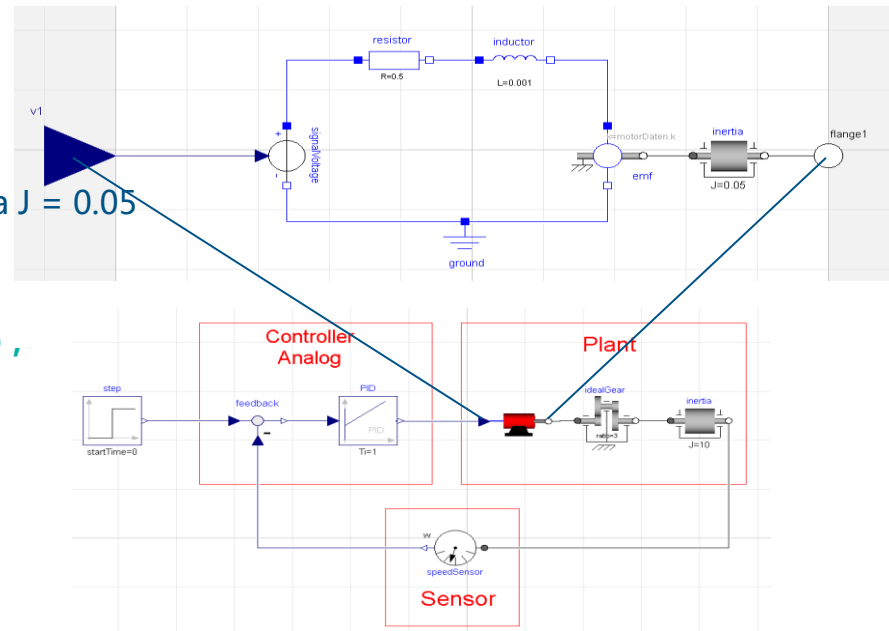
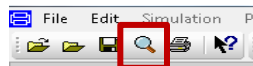
练习六：受控直流电机(1/2)

新建package Exercise6并在MSL中找到相应部件新建立电机模型

- ❖ Modelica.Electrical.Analog.Basic.Resistor R = 0.5
- ❖ Modelica.Electrical.Analog.Basic.Ground
- ❖ Modelica.Electrical.Analog.Sources.SignalVoltage
- ❖ Modelica.Electrical.Analog.Basic.Inductor L = 0.001
- ❖ Modelica.Electrical.Analog.Basic.EMF K = 1
- ❖ Modelica.Mechanics.Rotational.Components.Inertia J = 0.05

引用用电机模型建立闭环实验，并加入PID，idealGear，feedback反馈和速度传感器speedSensor和阶跃信号Step，可以使用Dymola的查找功能

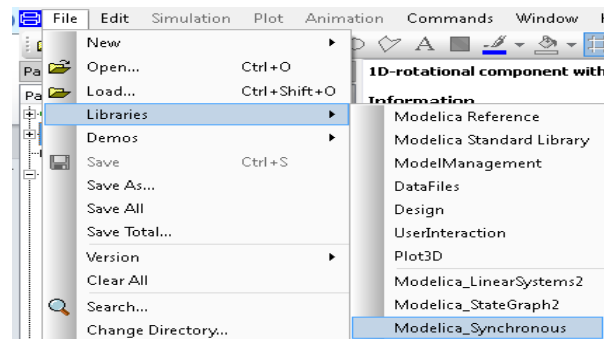
- ❖ Step height = 1
- ❖ Gear Ratio = 3
- ❖ Inertia J = 10
- ❖ PID k = 15
- ❖ PID Ti = 1
- ❖ PID Td = 0.001



练习六：受控直流电机(2/2)

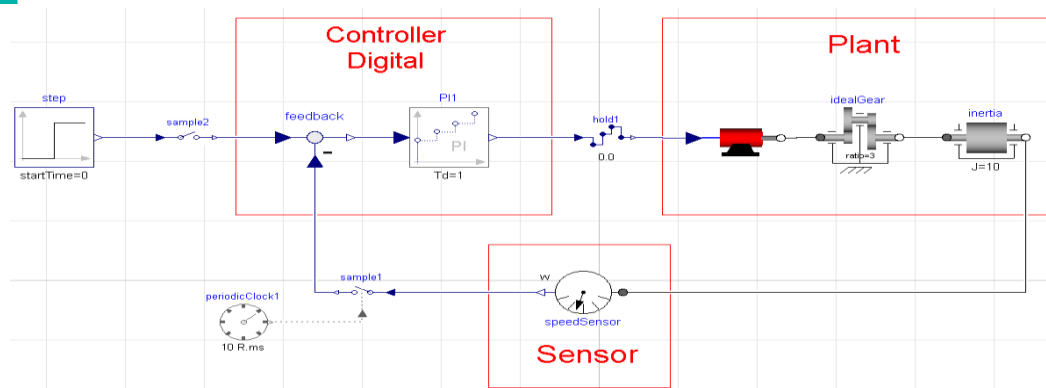
通过复制之前连续模型新建离散模型，加入Modelica_Synchronous库部件将原有控制器变为数字控制器

- ❖ Modelica_Synchronous.RealSignals.Periodic.PI
- ❖ Modelica_Synchronous.RealSignals.Sampler.Sample
- ❖ Modelica_Synchronous.RealSignals.Sampler.SampleClocked
- ❖ Modelica_Synchronous.RealSignals.Sampler.Hold
- ❖ Modelica_Synchronous.ClockSignals.Clocks.PeriodicExactClock

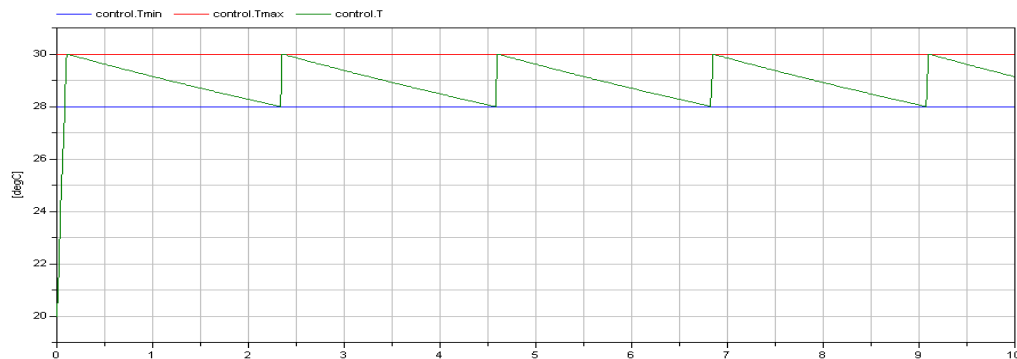
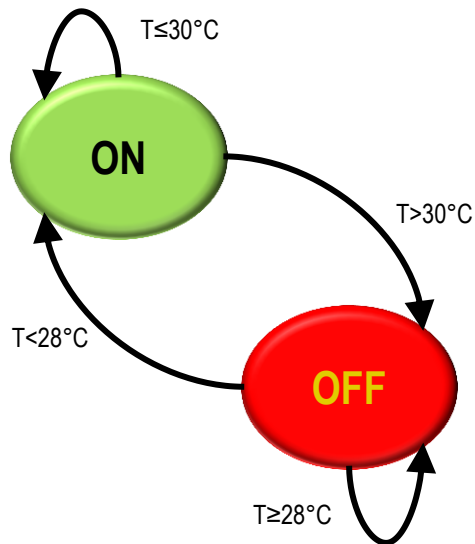


用以下参数进行仿真并对比连续控制器和数字控制器仿真结果

- ❖ Clock=10
- ❖ PI.K = 15
- ❖ PI.Td = 1



练习七：温度控制(1/6)



练习七：温度控制(2/6)

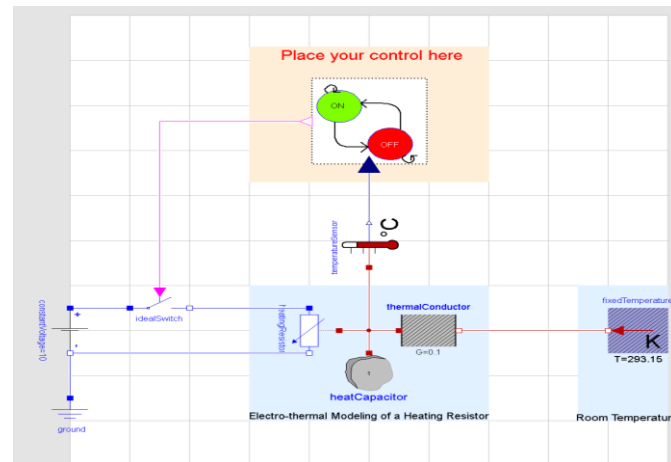
实验场景包含一个带有加热电阻和开关的电路。电阻产生的热通过热传导(Conduction)的方式传到20°恒温的外部空间

这个练习的目的是开发一个控制模块利用控制电路开关来维持电阻温度在27°C和30°C之间。控制器有以下的要求：

- ❖ 如果电阻温度 $T_r > 30^\circ\text{C}$, 电路打开
- ❖ 如果电阻温度 $T_r < 27^\circ\text{C}$, 电路闭合
- ❖ 如果电阻温度 $27^\circ\text{C} < T_r < 30^\circ\text{C}$, 电路维持原有状态

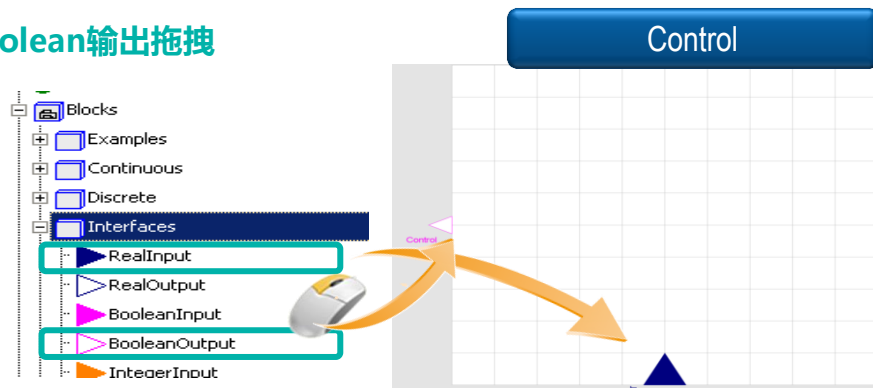
新建package Exercise7并在MSL中找到相应部件新建立受控电路模型

- ❖ Modelica.Electrical.Analog.Sources.ConstantVoltage ($V=10$)
- ❖ Modelica.Electrical.Analog.Ideal.IdealClosingSwitch
- ❖ Modelica.Electrical.Analog.Basic.HeatingResistor
- ❖ Modelica.Thermal.HeatTransfer.Components.HeatCapacitor ($C=1$)
- ❖ Modelica.Thermal.HeatTransfer.Components.ThermalConductor ($G=0.1$)
- ❖ Modelica.Thermal.HeatTransfer.Sources.FixedTemperature ($T=293.15$)
- ❖ Modelica.Thermal.HeatTransfer.Sensors.TemperatureSensor

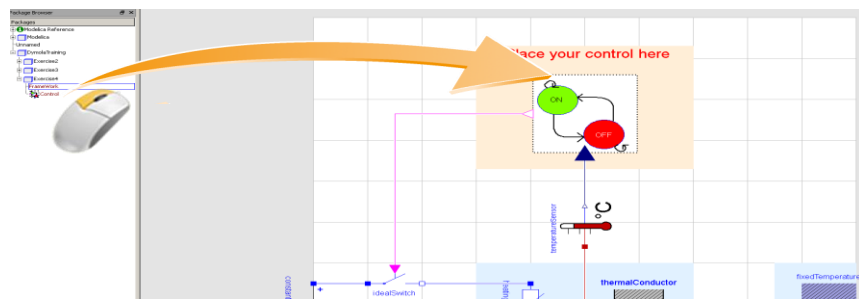


练习七：温度控制(3/6)

新建空白模型作为控制器并在MSL中找到Real输入和Boolean输出拖拽入模型



为控制器绘制图标并拖入之前建立的受控电路模型并分别连接电路开关和温度计



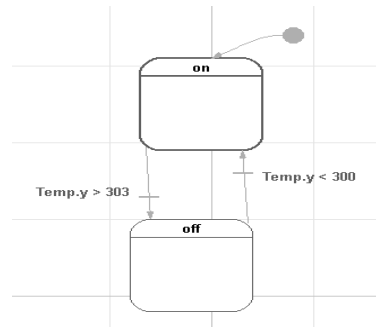
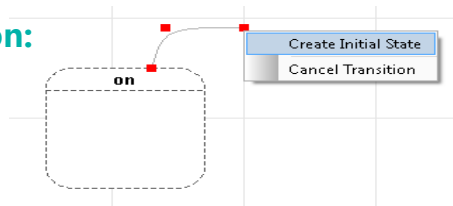
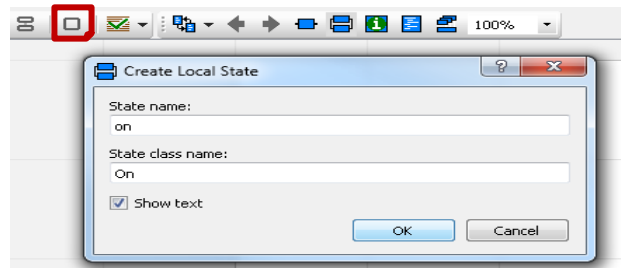
练习七：温度控制(4/6)

重新切换到控制器模型，如图点击“Create Local State” 创造新本地状态并命名“On”，点击显示文本

点击状态边界并按住鼠标移开到空白处，再次点击创造初始状态

创造另外一个状态“Off”并定义状态转移条件Transition:

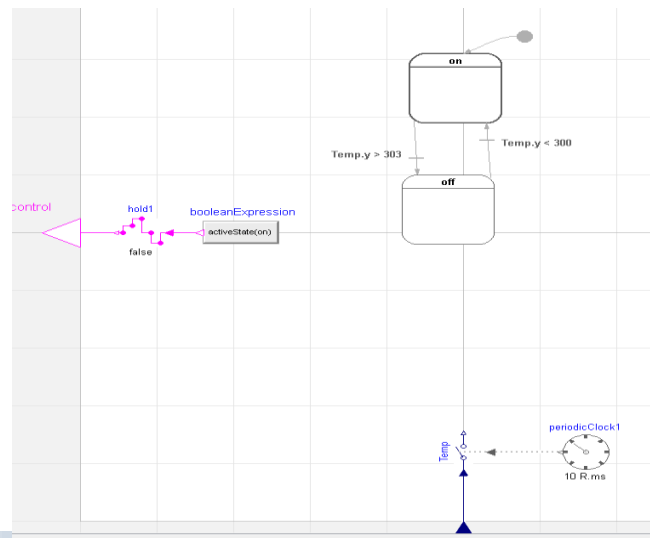
- ❖ 从on到off: Temp.y>303; Reset
- ❖ 从off到on: Temp.y<300; Reset



练习七：温度控制(5/6)

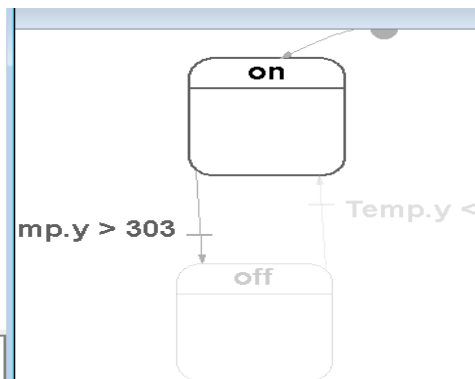
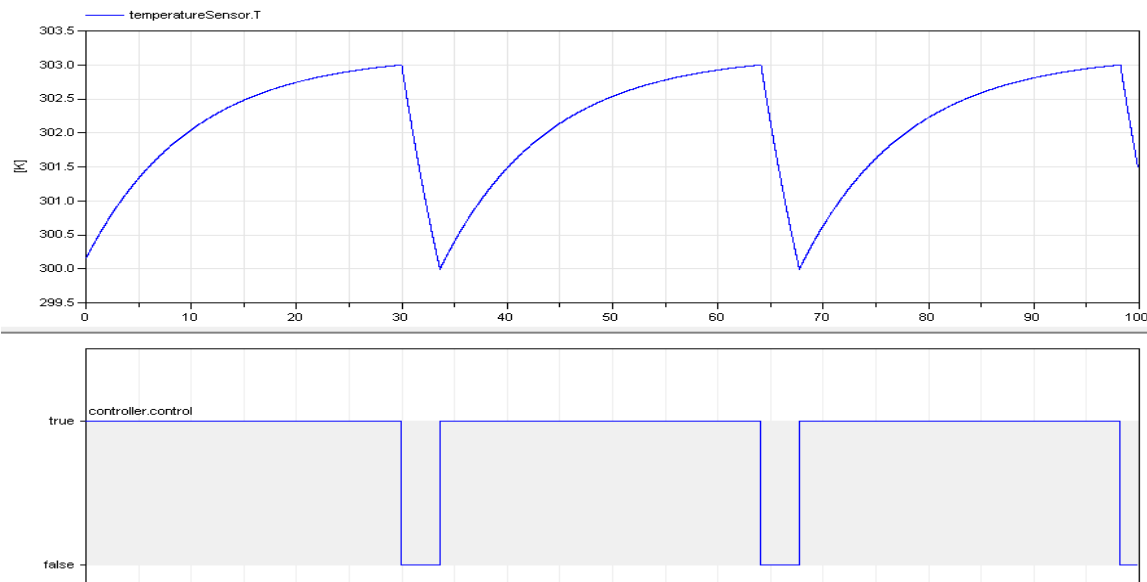
导入Modelica_Synchronous库: File/Libraries/Modelica_Synchronous

- ❖ Modelica_Synchronous.RealSignals.Sampler.SampleClocked 并命名为 “Temp”
- ❖ Modelica_Synchronous.BooleanSignals.Sampler.Hold.
- ❖ Modelica.Blocks.Sources.BooleanExpression填入 “activeState(on)”
- ❖ Modelica_Synchronous.ClockSignals.Clocks.PeriodicExactClock sampleTime = 10.



练习七：温度控制(6/6)

导入Modelica_Synchronous库: File/Libraries/Modelica_Synchronous
设置仿真时间100s，查看温度和控制器变量曲线
打开Diagram界面看状态机变化动画



Arrays 数组(1/5)

数组可被看作是同样类型数值的集合，Modelica数组可是多维的

- ❖ 图表中C代表了Modelica任意的类，如Real, Integer, Boolean, String和enumeration类.
- ❖ k, n, m, n1,n2是Integer的子类
- ❖ E代表的是enumeration类

Modelica form 1	Modelica form 2	# dimensions	Designation	Explanation
C r;	C r;	0	Scalar	Scalar
C[n] v;	C v[n];	1	Vector	n – Vector
C[E] v;	C v[E];	1	Vector	Vector index by enumeration type E
C[n, m] M;	C M[n, m];	2	Matrix	n x m Matrix
C[n1, n2, ..., nk] M;	C M[n1, n2, ..., nk];	k	Array	Array with k dimensions (k>=0)

Arrays 数组(2/5)

基本的数组操作

❖ 声明

```
Real v[3]={1,2,3};
```

$$v = (1 \quad 2 \quad 3)$$

```
Real m2[2,3] = [11,12,13;21,22,23];
```

$$m1 = m2 = \begin{bmatrix} 11 & 12 & 13 \\ 21 & 22 & 23 \end{bmatrix}$$

```
Real m1[2,3] = {{11,12,13},{21,22,23}};
```

❖ 构造

```
Real v[6]=1:6;
```

$$v = (1 \quad 2 \quad 3 \quad 4 \quad 5 \quad 6)$$

```
Real v1[4]=1:3:10;
```

$$v1 = (1 \quad 4 \quad 7 \quad 10)$$

```
Real[ :,:] H=({1/(i+j-1) for i in 1:3} for j in 1:3);
```

❖ 抽取

```
Real H1[2,2]=H[{2,3},{1,2}];
```

$$H1 = \begin{bmatrix} \frac{1}{2} & \frac{1}{3} \\ \frac{1}{3} & \frac{1}{4} \end{bmatrix} \quad H = \begin{bmatrix} 1 & \frac{1}{2} & \frac{1}{3} \\ \frac{1}{2} & \frac{1}{3} & \frac{1}{4} \\ \frac{1}{3} & \frac{1}{4} & \frac{1}{5} \end{bmatrix}$$

Arrays 数组(3/5)

基本的数组操作：加法，减法，标量和矩阵乘法

```
parameter Real s=3;
parameter Real A[3,3]=zeros(3,3);
Real B[3,3];
Real C[3,3];
Real D[3,3];
parameter Real v[1,3]={1:3};

equation

    B=s*A;           //multiplication of a matrix by a scalar
    C=A+B;           //matrix sum
    D=A*B;           //matrix product
```

Arrays 数组(4/5)

更多数组操作：加法，减法，标量和矩阵乘法

	Modelica	Explanation
Size	ndims(A)	Returns the number of dimensions k of array expression A, with $k \geq 0$.
	size(A,i)	Returns the size of dimension i of array expression A where i shall be > 0 and $\leq \text{ndims}(A)$.
	size(A)	Returns a vector of length ndims(A) containing the dimension sizes of A.
Constructors	diagonal(v)	Returns a square matrix with the elements of vector v on the diagonal and all other elements zero.
	identity(n)	Returns the n x n Integer identity matrix, with ones on the diagonal and zeros at the other places.
	zeros(n1,n2,n3,...)	Returns the n1 x n2 x n3 x ... Integer array with all elements equal to zero ($n_i \geq 0$).
	ones(n1,n2,n3,...)	Returns the n1 x n2 x n3 x ... Integer array with all elements equal to one ($n_i \geq 0$).
Reduction	min(A)	Returns the smallest element of array expression A.
	sum(A)	Returns the scalar sum of all the elements of array
	product(A)	Returns the scalar product of all the elements of array expression A.
Algebra	transpose(A)	Permutates the first two dimensions of array A.
	outerProduct(v1,v2)	Returns the outer product of vectors v1 and v2 ($= \text{matrix}(v1) * \text{transpose}(\text{matrix}(v2))$).
	sym metric(A)	Returns a matrix where the diagonal elements and the elements above the diagonal are identical to the corresponding elements of matrix A and where the elements below the diagonal are set equal to the elements above the diagonal of A.
	cross(x,y)	Returns the cross product of the 3-vectors x and y, i.e. $\text{cross}(x,y) = \text{vector}(\{x[2]*y[3]-x[3]*y[2], x[3]*y[1]-x[1]*y[3], x[1]*y[2]-x[2]*y[1]\})$.
	skew(x)	Returns the 3 x 3 skew sym metric matrix associated with a 3-vector, i.e., $\text{cross}(x,y) = \text{skew}(x)*y$, $\text{skew}(x) = [0, -x[3], x[2]; x[3], 0, -x[1]; -x[2], x[1], 0]$.

Arrays 数组(5/5)

模型内组件的组合

❖ Modelica允许声明类（接口、模型等）的数组，大大提高处理同一模型内部多个同类的组件

```
model ULine "Lossy RC Line"
import A = Modelica.Electrical.Analog;
A.Interfaces.Pin p;
A.Interfaces.Pin n;
parameter Integer N(final min=1) = 1 "Number of lumped segments";
parameter Real r=1 "Resistance per meter";
parameter Real c=1 "Capacitance per meter";
parameter Real L=1 "Length of line";
protected
parameter Real Re=r*L/(N + 1);
A.Basic.Resistor R[N + 1] (R=vector([Re/2; fill(Re, N - 1); Re/2]));
A.Basic.Capacitor C[N] (each C=c*L/N);
A.Basic.Ground g;
equation
for i in 1:N loop
connect(R[i].n, R[i + 1].p);
connect(R[i].n, C[i].p);
connect(C[i].n, g);
end for;
connect(p, R[1].p);
connect(R[N + 1].n, n);
end ULine;
```

